



University
of Basel

Chapter 1: Introduction

Foundations of Distributed Systems

Fall Semester 2026

Prof. Dr. Florina Ciorba

Prof. Dr. Heiko Schuldt

Prof. Dr. Christian Tschudin

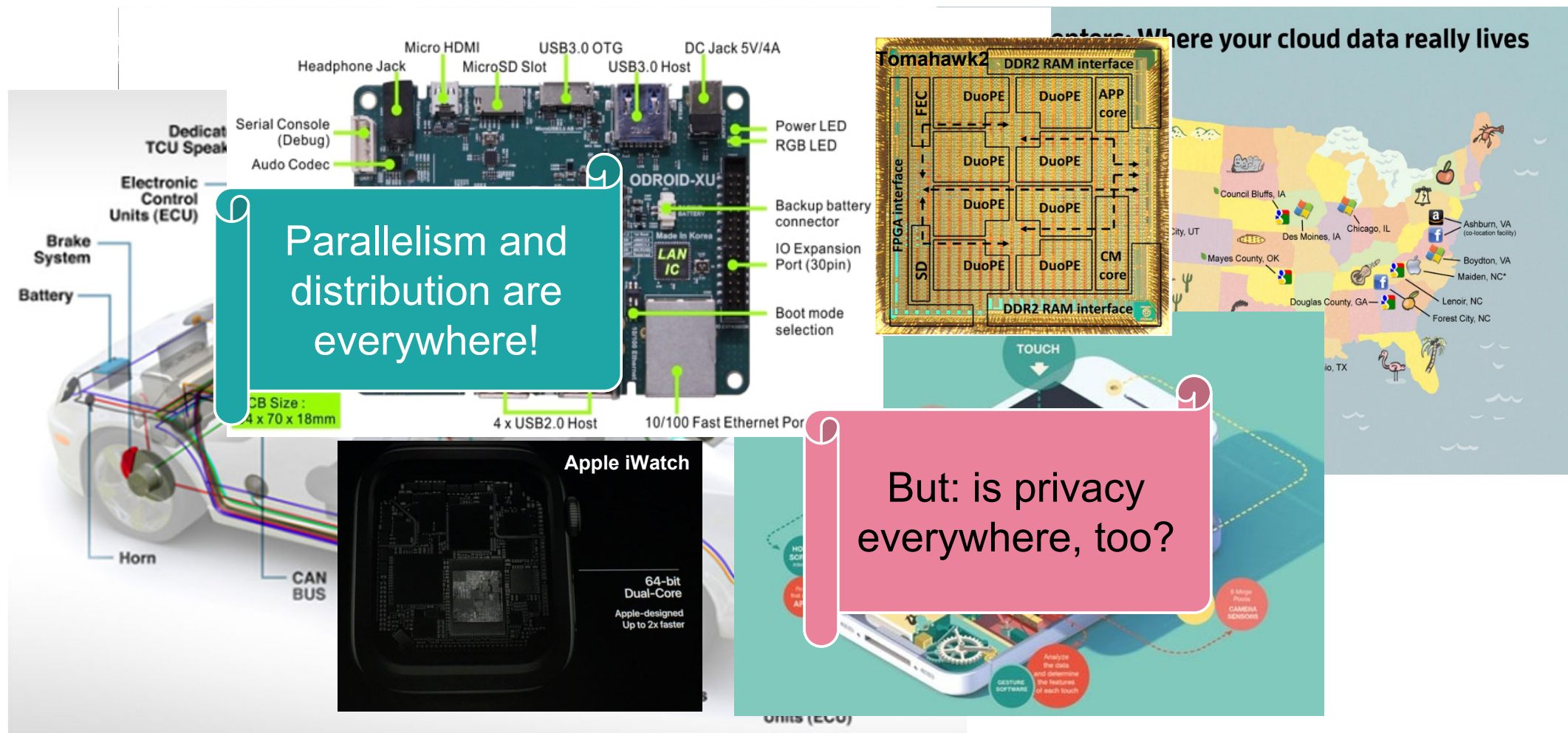
Prof. Dr. Isabel Wagner



Overview of Chapter 1

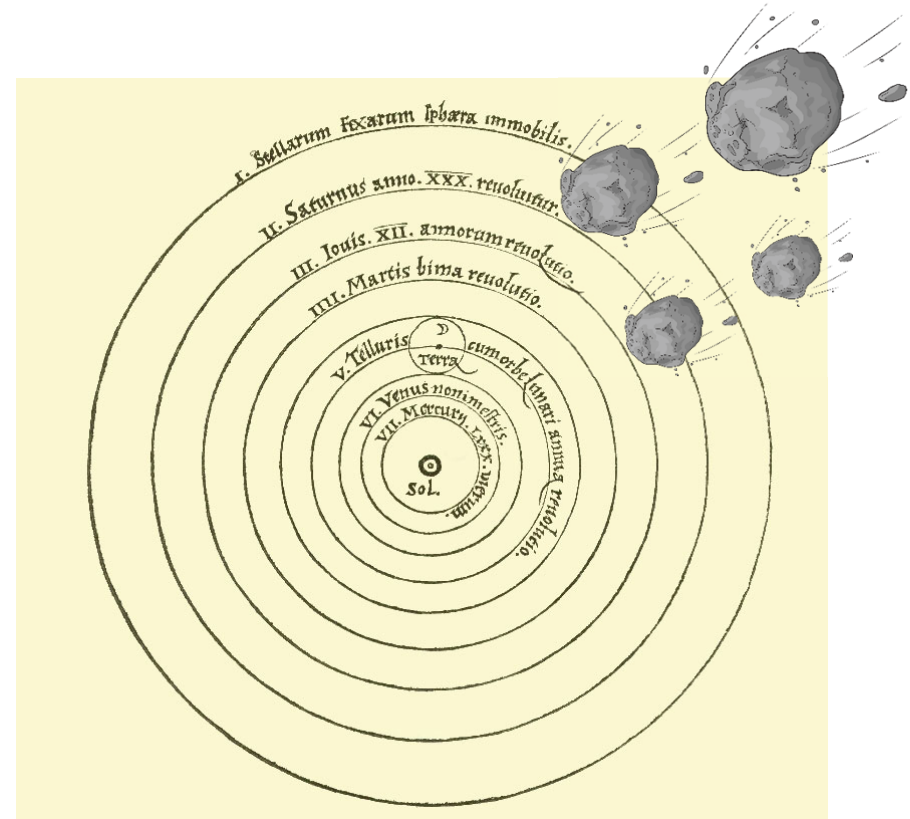
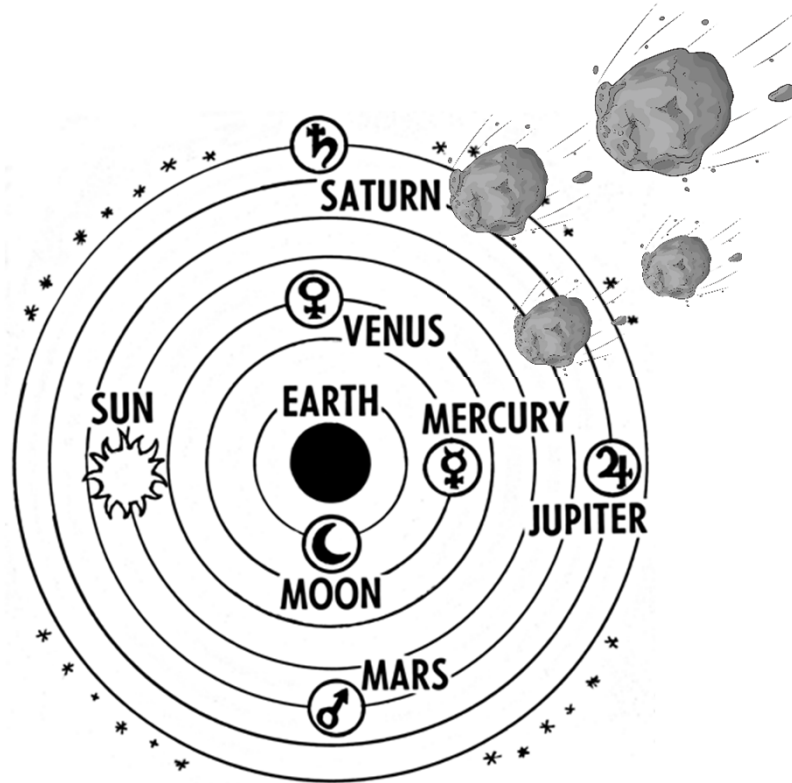
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Data Perspective |
| 1.4 | ... from a Processing Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |

1.1 What are Parallel and Distributed Systems?



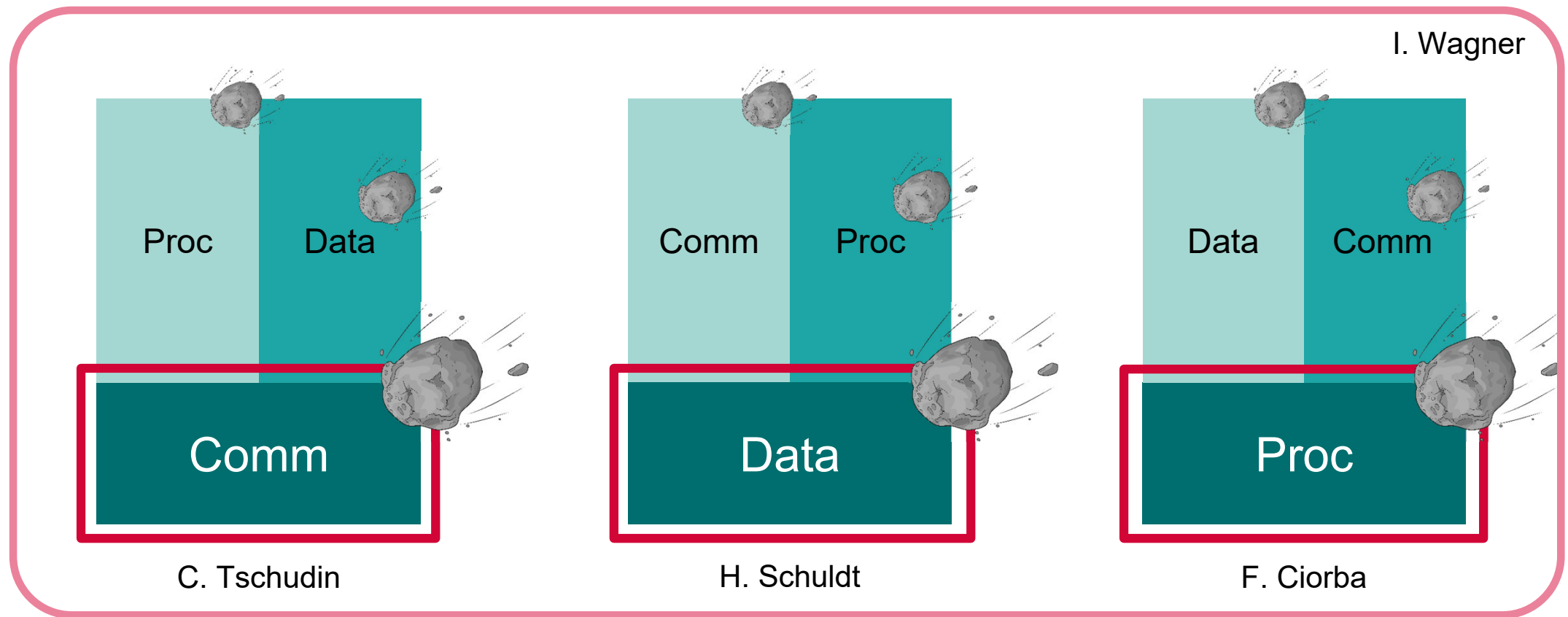
Ptolemaic vs Copernican View of the World

i.e., Communication/Processing/Data (comm/proc/data)



- Privacy can be considered as a swarm of Asteroids challenging all planets

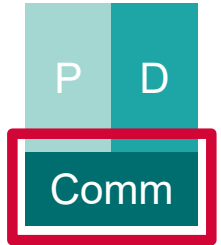
Where the Sun is (or the Earth) and where the Threat comes in



Overview of Chapter 1

-
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Data Perspective |
| 1.4 | ... from a Processing Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |
-

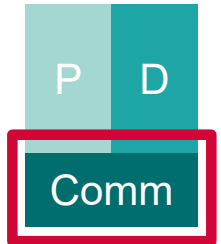
1.2 What are parallel and distributed systems, from a Communications Perspective?



Distributed computer systems

- did not exist 50 years ago *because computer **networks** did not exist, back then*
- did not work well, 40 years ago *lack of **protocol** design theory*
- did require hefty servers, 30 years ago ***peer-to-peer** was not invented yet*
- did not scale well, 20 years ago *the **cloud** did not exist yet*
- found themselves in a centrality black hole, 10 years ago ***economic** deficits, “enshittification”*

Context: “Enshittification”



Term coined and popularized since 2022 by Cory Doctorow, “word of the year” in 2023+2024

Before: “crapification of software”, “platform decay”

Process:

- a) offer a service to users, at loss (to gain new users) → now they are locked in
 - b) offer access to userbase to suppliers (to gain new suppliers), at loss → now they are locked in
 - c) ask money from the user
 - d) ask money from the suppliers
 - e) “twiddling”: continual adjustment of parameters to improve profits, quality does not matter
- See Facebook, Google Search, Netflix, Airbnb, Amazon, Twitter, Uber ...

1.2 What are parallel and distributed systems, from a Communications Perspective?

Distributed computer systems

- did not exist 50 years ago
- did not work well
- did require hefty
- did not scale well,
- found themselves in

tremendous progress over 50 years
 (your powerful smartphone
 is just the tip of an iceberg,
 sits on the top of a humongous
 technology pyramid)

not exist, back then

ol design theory

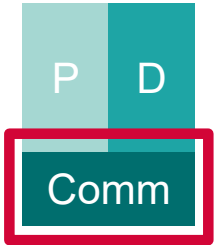
not invented yet

the **cloud** did not exist yet

hole, 10 years ago

economic deficits, “enshittification”

1.2 **What are** parallel and distributed systems, from a Communications Perspective?



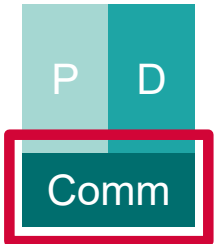
tremendous progress over 50 years
(your powerful smartphone
is just the tip of an iceberg,
sits on the top of an humongous
technology pyramid)

This means for your education:

- ways too much innovation and knowhow to absorb
- scientific shakeout is still ongoing
("What will survive the test of time?")
- **foundations** will not go away, we have to work out the big lines

"So what you say is: **it all started with computer networks**, right?"

1.2 What are parallel and distributed systems, from a Communications Perspective?



Distributed “Computers” DID exist, 80 years ago



Distributed “computers” at NASA Jet Propulsion Laboratory, starting in 1936. Calculated launch windows, trajectories, fuel consumption of rockets...

Other human computers received computations problems and sent back results by mail!

Some were making mistakes and others might be interrupted and not complete their task!

➡ **Central** steering and funding: “command-control” context.
>> Computer Science is about automating ... also this

Going back to the Basics, Prepare for the Future

P D

Comm

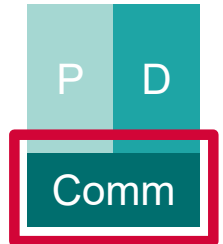
In FDS.**Comm** you will learn about **distribution concepts (the basics)** useful for both centralized (cloud) and decentralized settings:

- **Concurrency** and why it is hard, theoretical limits of distributed systems, failure modes – “**physics of computers**”
- How to work *without* precise time, how to **scale**, how to persist data in a **crash-resilient** way, how to handle **malicious actors**, how to **coordinate** work in a distributed system without bottleneck **consensus**



main take-home message

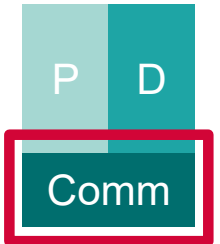
focus is on “Distribution”
although area label is “Comm”



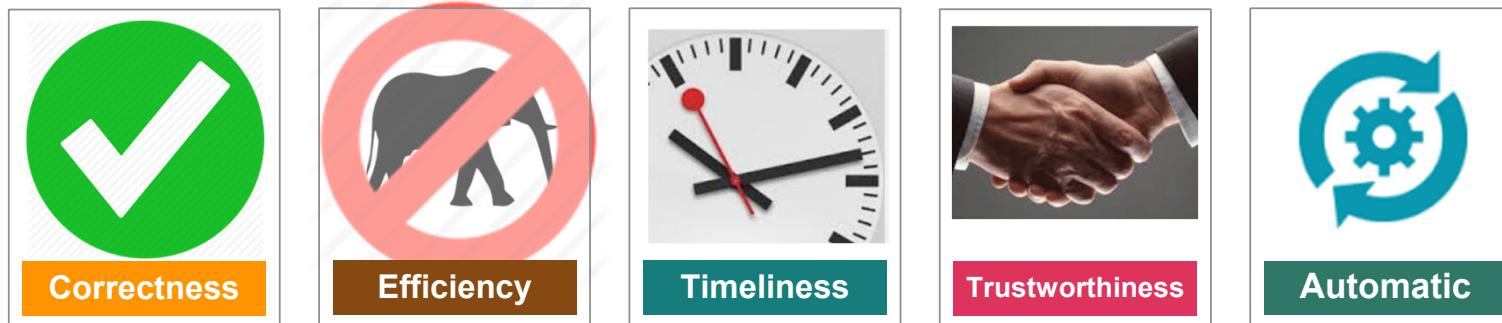
Distributed computer systems made tremendous progress, as they

- did not exist **50 years ago** *because computer networks did not exist, back then*
- did not work well, **40 years ago** *lack of protocol design theory*
- did require hefty servers, **30 years ago** *peer-to-peer was not invented yet*
- did not scale well, **20 years ago** *the cloud did not exist yet*
- found themselves in a centrality black hole, **10 years ago** *economic deficits, “enshittification”*

From a Communication Perspective: Goals & Challenges of Distributed Systems



Goals



Challenges



The current ‘

- Is the “AI doom

‘In 6-12 months ... a swarm capable of taking over the world with a persistent botnet causing hundreds of billions of damage’ – Dario Amodei of Anthropic, 12 September 2023

Forbes

Subscribe: Less than \$2/wk

INNOVATION > VENTURE CAPITAL

Bots Now Outnumber Humans Online And The Internet Was Never Built For This

By [Josipa Majic Predin](#), Contributor. © Founder & Advisor on Emerging Tech....

[Follow Author](#)

Published Jun 04, 2026, 08:49am EDT

Just yesterday [Cloudflare CEO Matthew Prince posted on X](#) that automated bot traffic had crossed a threshold no one in the industry expected this soon: [for the first time in the internet’s history, machines now generate more web traffic than people](#). [Cloudflare’s Radar dashboard](#) puts bots at 57.5% of all HTTP requests to HTML content, humans at 42.5%. Prince had predicted the crossover by end of 2027, but we got there eighteen months early.

June 2026

The Guardian

Eur ▾

port Culture Lifestyle

sis Ukraine Environment Science Global deve

igence)

ally wipe out humanity - spell out the risks

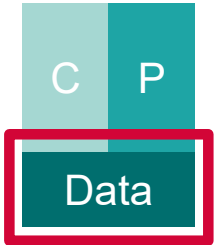
and counterclaims about the risks and the pace of AI development

[Milmo and Aisha Down](#)

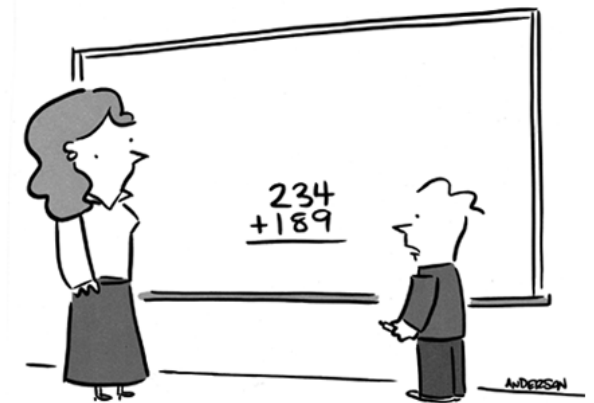
Overview of Chapter 1

-
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Data Perspective |
| 1.4 | ... from a Processing Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |
-

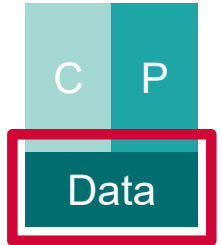
1.3 What are Distributed Systems? An Answer from a Data Perspective



- A Distributed System is a system in which (at least) the data layer is distributed (either logically –on the same computer system– or physically – across several systems connected via a network)
- Distributed Data Management is particularly relevant in the context of **Big Data**
 - “Data is all around”
 - The term Big Data is actually not new ...
 - ... but what people conceive as Big Data has changed significantly over the past decades



“Big Data” back in 1975



Example: Data management at the Dutch telecom PTT

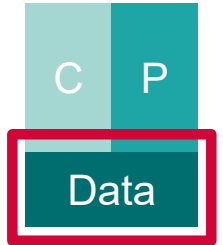
- Data size
 - 5 Mio. phone numbers
 - 4.5 Mio. subscribers
 - 0.275 Mio. work orders
 - 0.15 Mio. streets
 - 3'500 cities
- Data Access
 - 110'000 bills per day
 - 105'000 inquiries per day
- Updates
 - 110'000 payments per day
 - 10'000 subscriber updates per day
 - 2 DB copies (backup) per day
 - 1 reorganization per week (35'000 records)



Source: “Database Experience in the Netherlands PTT” (VLDB, 1975):



“Big Data” today



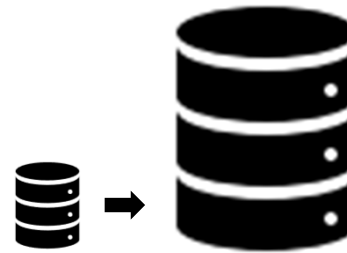
- Facebook
 - approx. 350 million of new photos per day
(numbers from 2020, source: <https://www.omnicoreagency.com/facebook-statistics/>)
 - More than 300 petabytes of data (source: <https://blog.wishpond.com/post/115675435109/40-up-to-date-facebook-facts-and-stats>)
 - approx. 100 million hours of video are watched every day
- Instagram (numbers from 2020, source <https://www.omnicoreagency.com/instagram-statistics/>)
 - 50+ billion photos in total
 - approx. 100+ million new photos per day
- WhatsApp
 - Over 100 billion messages per day
- YouTube (numbers from 2022, source: <https://www.omnicoreagency.com/youtube-statistics/>)
 - In total, 5+ billion videos have been shared (2020)
 - approx. 500 hrs. of new videos uploaded per minute
 - Total Daily Hours of Video Watched: 1 billion



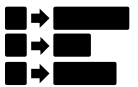
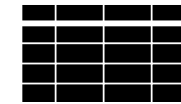
And What is Big Data?

- Characterized by the “3 V’s”

- **Volume** (very large quantities of data)



- **Variety** (heterogeneity of data, data integration, multiple data models, semantic interoperability)



- **Velocity** (dynamics, “fast” data, continuous data streams, time-series, complex event detection and processing)



- Extended by additional V’s:

- **Veracity**

- **Validity**

- **Variability**

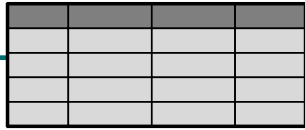
- **Visualization**

- **Volatility**

- **Value ...**

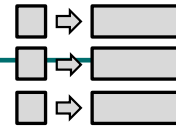
Evolution of Data Models

Relational



- Table representation
- Structured data in a strict schema
- Transactional workloads

Key-Value



- Array-like data
- High performance and scalability
- Allows storing arbitrary values

Document

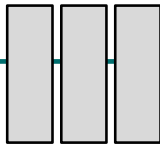
C P

Data



- Collection of documents
- Unstructured data / following no schema
- Nested structures

Analytical



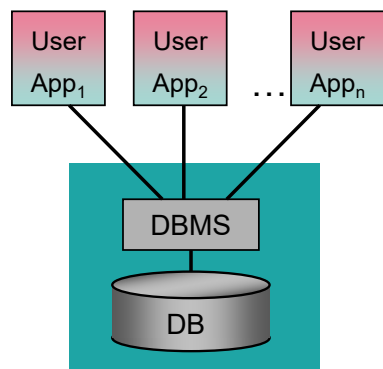
- Comes in various shapes, e.g., based on the relational model
- All items of a column are stored together

Labeled-Property Graph



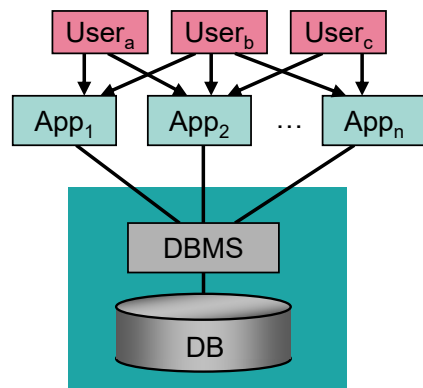
- Data is represented using nodes and edges
- Good for storing and querying relationships

(Simplified) Evolution of Database Architectures



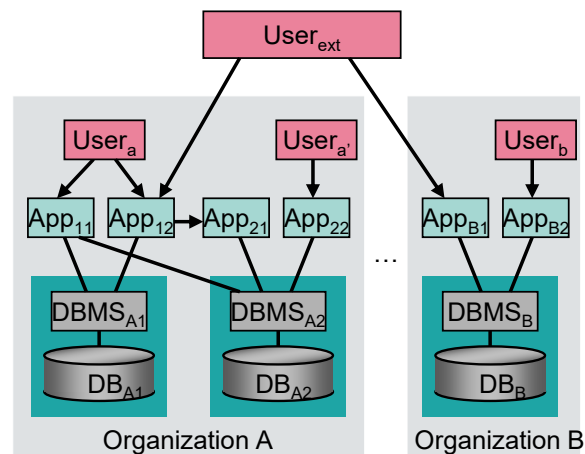
Two-tier
Architecture

Centralized
Data Management



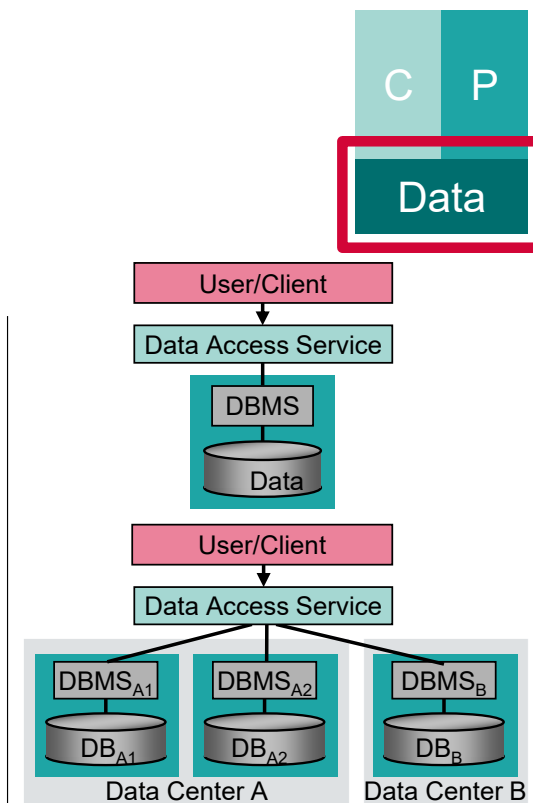
Three-/multi-tier
Architecture

Centralized
Data Management



Three-/Multi-Tier
Architecture

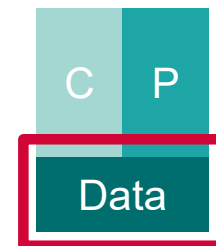
Distributed
Data Management



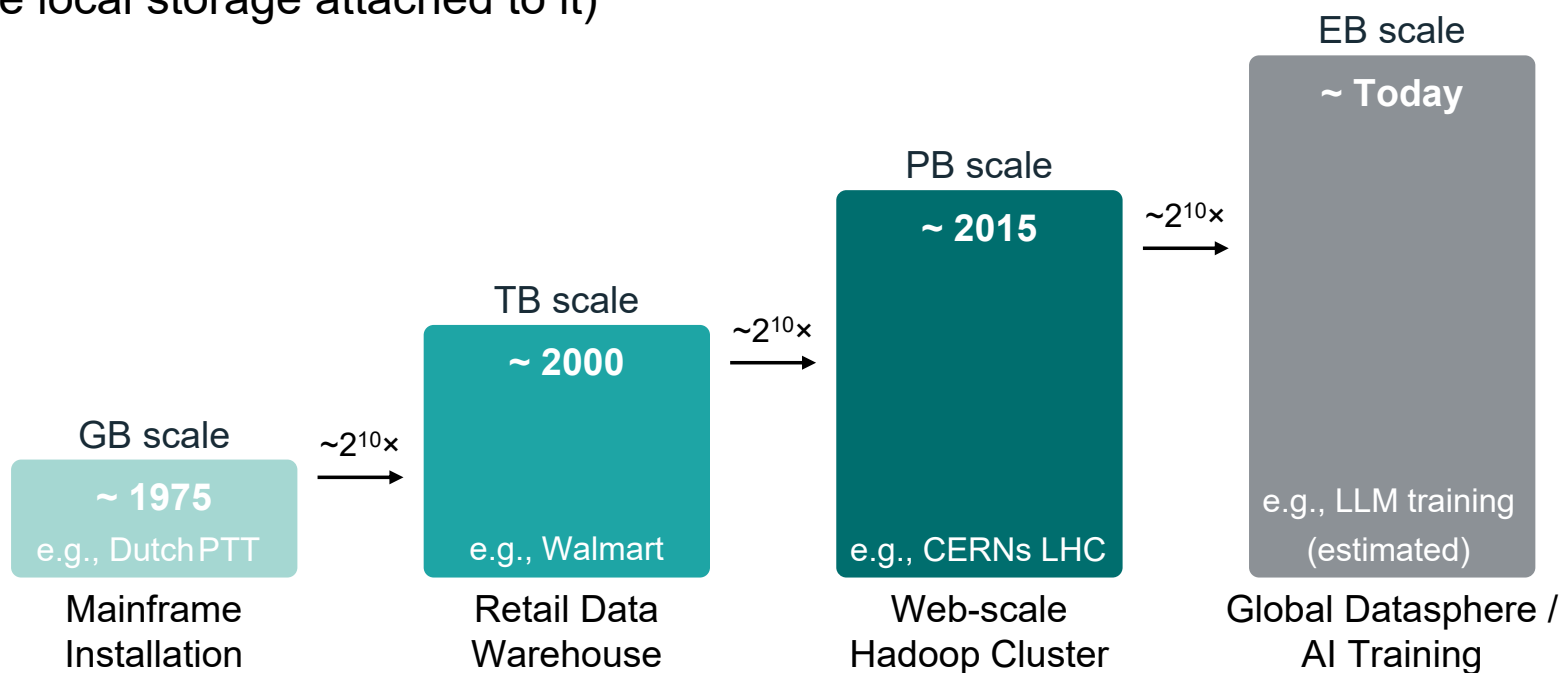
Appears like: Two-Tier,
Centralized Architecture

Actually is: Two-/Multi-Tier
Distributed
Data Management

Big Data: the “Invariant”

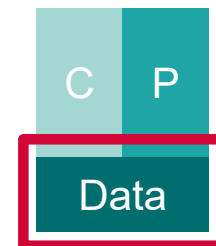
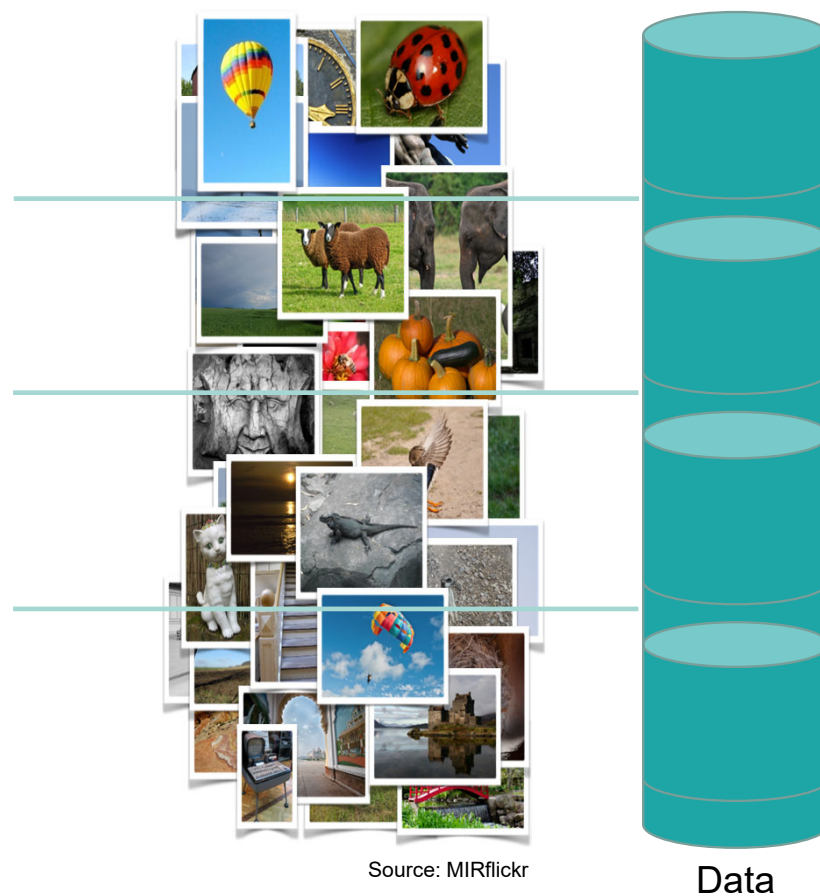


- Independent of the sheer size of Big Data (depending on the interpretation at the time), there is one important invariant:
 - The volume of data is in most cases **too big to be managed on a single computer** (and the local storage attached to it)

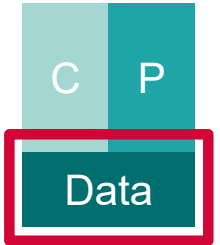


Explicit Distribution of Data (Data Sharding)

- Solution: split data in smaller chunks and distribute them across several computers (nodes in a distributed system)
- Distribution for
 - Data management
 - Data processing (partitioning and/or replication, for parallel access)

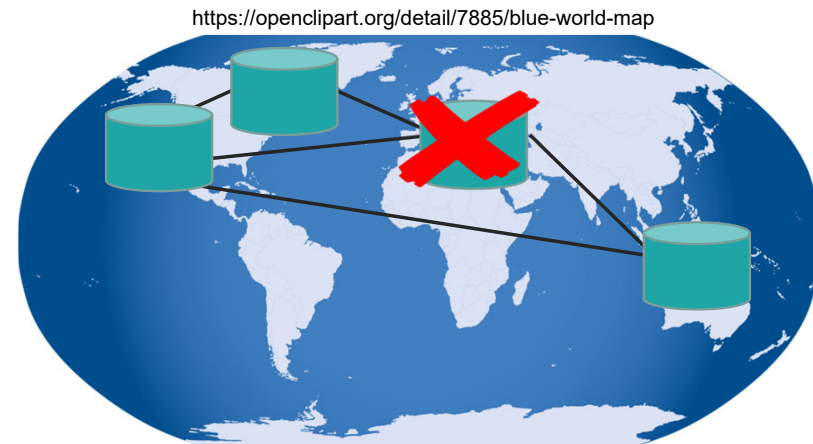


Data Distribution

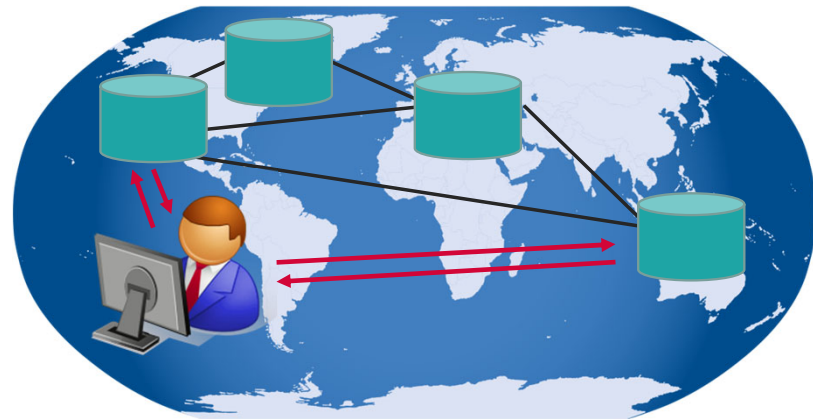


Further reasons for (global) data distribution

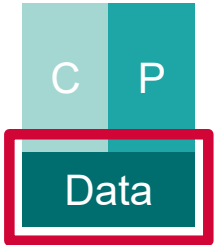
- **Fail-safe systems:** replicated data



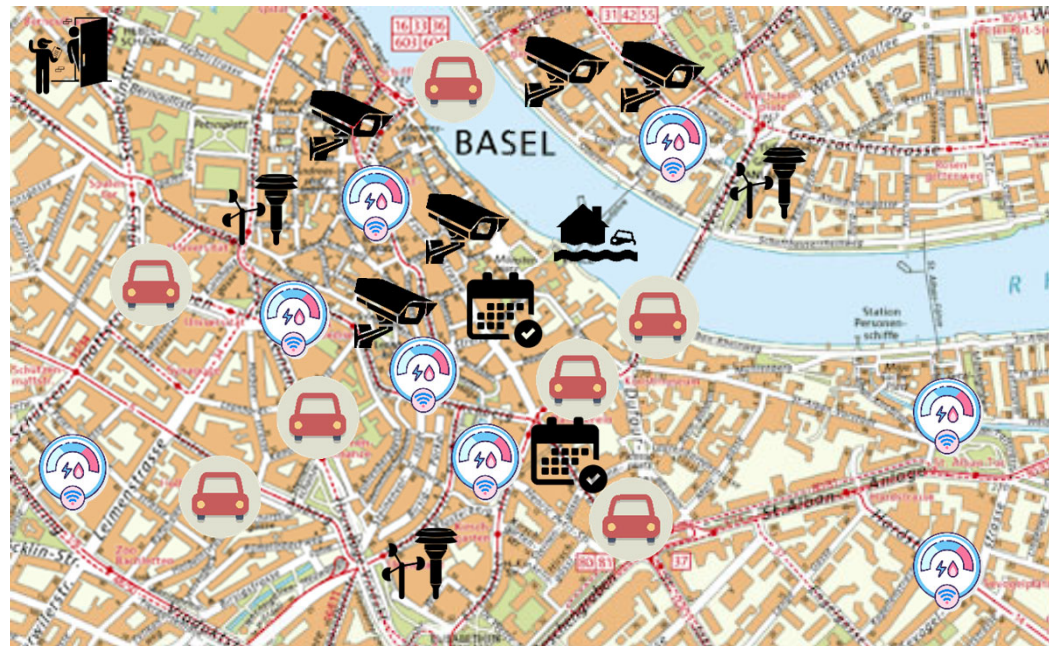
- **Efficient access:** make data accessible close to their users



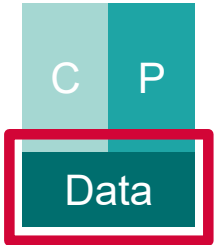
Integration of Data



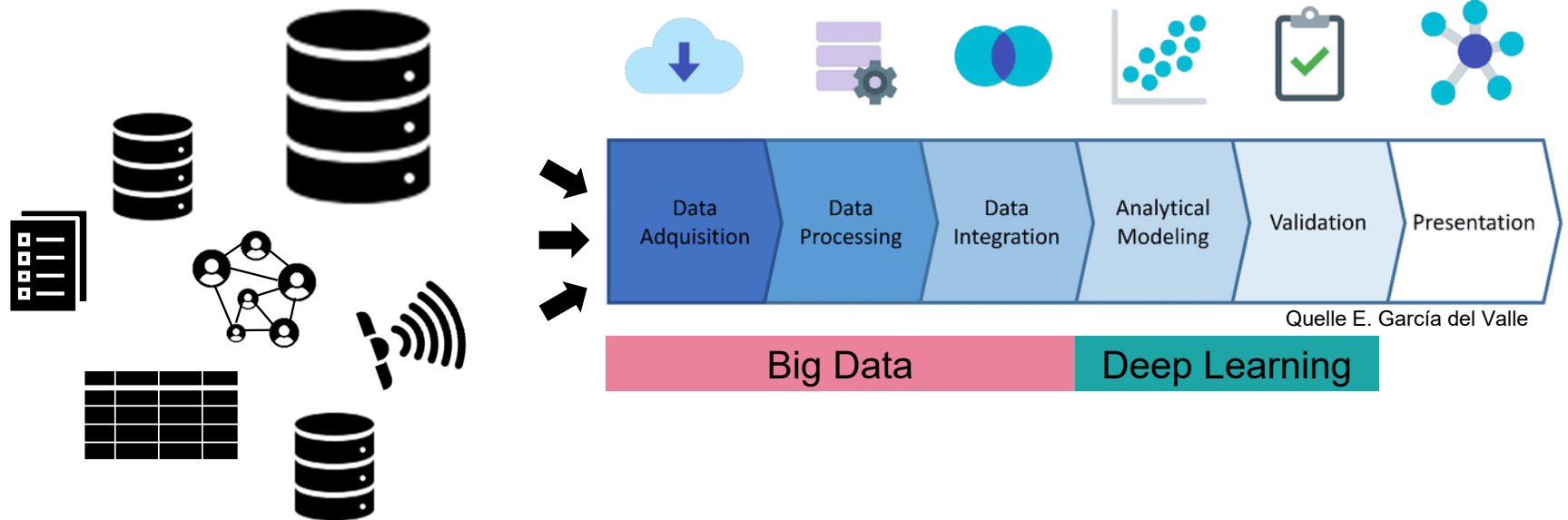
- However, in some cases the distribution of data is not a solution, but rather part of a challenge
- Data collections that have evolved naturally and independently over the years
 - Challenge: Integrate heterogeneous data to jointly analyze it



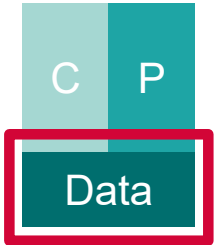
Big Data and Data Science



- Data Analytics is “only” the last step in the Data Science pipeline
- The first, essential part, needs to deal with Big Data, and in particular **distributed data**
 - Volume
 - Heterogeneity
 - Velocity



From a Data Perspective: Goals & Challenges of Distributed Systems



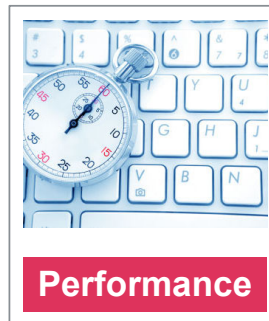
- Goals



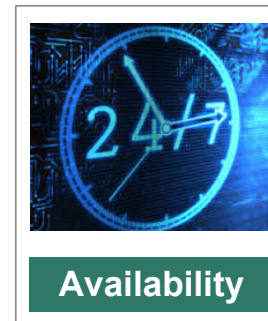
Scalability



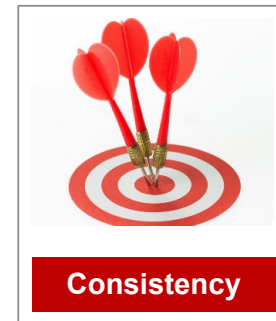
Scalability



Performance

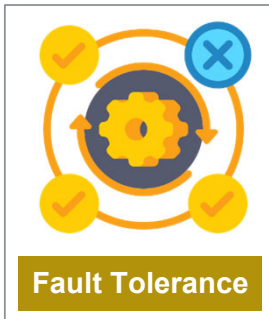


Availability

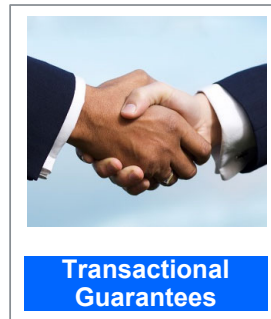


Consistency

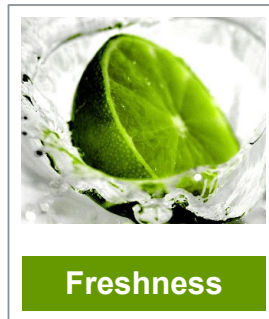
- Challenges



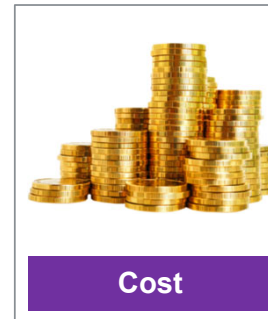
Fault Tolerance



Transactional
Guarantees



Freshness

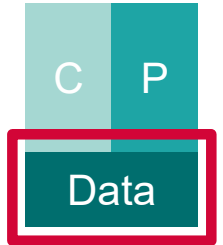


Cost



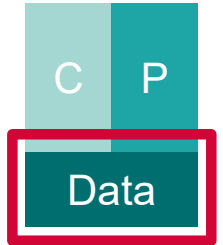
Adaptivity

From a Data Perspective: Goals of Distributed Systems



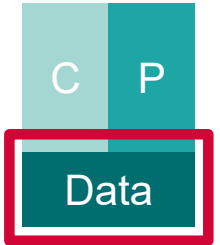
- Using a distributed system to manage data allows to cope with huge volumes and to **scale** the data management layer together with the application demands
- Distributing data also allows to **parallelize** the access to data and/or to balance the load in the system and thus to increase the overall **performance** of systems
- At the same time, managing data at several sites may also increase the **availability** of data in case of replication, but raises the challenge to guarantee the **consistency** of data (make sure that all copies of an object have the same value)

From a Data Perspective: Challenges of Distributed Systems

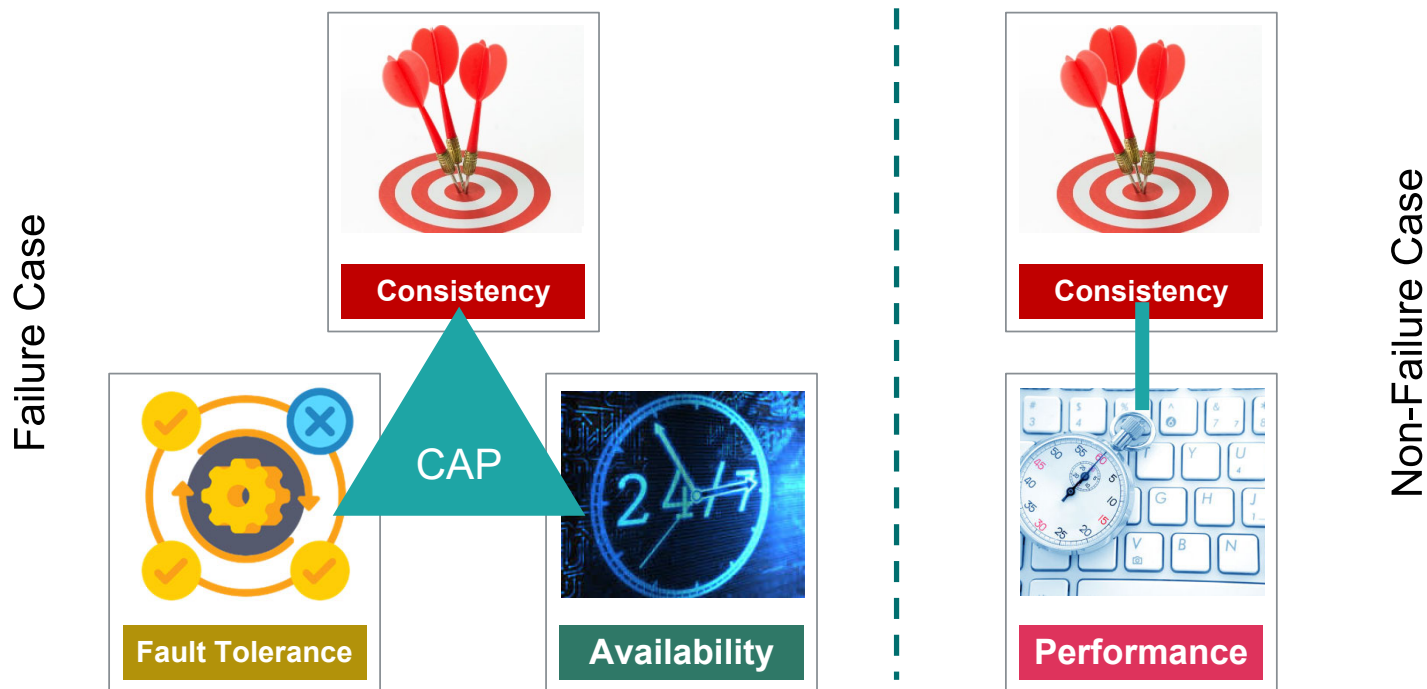


- Distributed Data Management comes with some major challenges
 - **Fault tolerance** – make sure that (partial) failures can be coped with correctly
 - **Transactional guarantees**: make sure that applications that span several sites respect the ACID properties of database transactions
 - **Data Freshness**: make sure that data, if available in several versions (freshness levels) is provided with at least the desired degree of freshness
 - **Cost**: provide all these features in a cost-optimized, economic way
 - **Adaptivity**: dynamically apply changes (e.g., to data distribution, coordination protocols, etc.) as the access characteristics to data changes
- Additional challenges: deal with
 - (structural/semantic) **heterogeneity**
 - **Dynamic changes** (e.g., schema evolution)
 - ...

Dependencies



- Some of the goals / challenges are mutually exclusive
 - CAP Theorem: only two out of **C**onsistency, **A**vailability, **P**artition Tolerance
 - More precisely: PACELC: **CAP** (in failure case) **E**lse **L**atency or **C**onsistency



From a Data Perspective: Goals & Challenges of Distributed Systems

C P

Data

Chapter 4.3: Data Replication

Chapter 4.1: Self-Organization



Scalability



Scalability



Performance



Availability



Consistency

(Chapter 3.5: Consensus)

Chapter 4.2: Consistency



Fault Tolerance



Transactional
Guarantees



Freshness



Cost



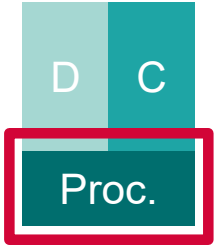
Adaptivity

Overview of Chapter 1

-
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Data Perspective |
| 1.4 | ... from a Processing Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |
-

1.4 What are Distributed Systems?

An Answer from the **Processing of Computations** Perspective



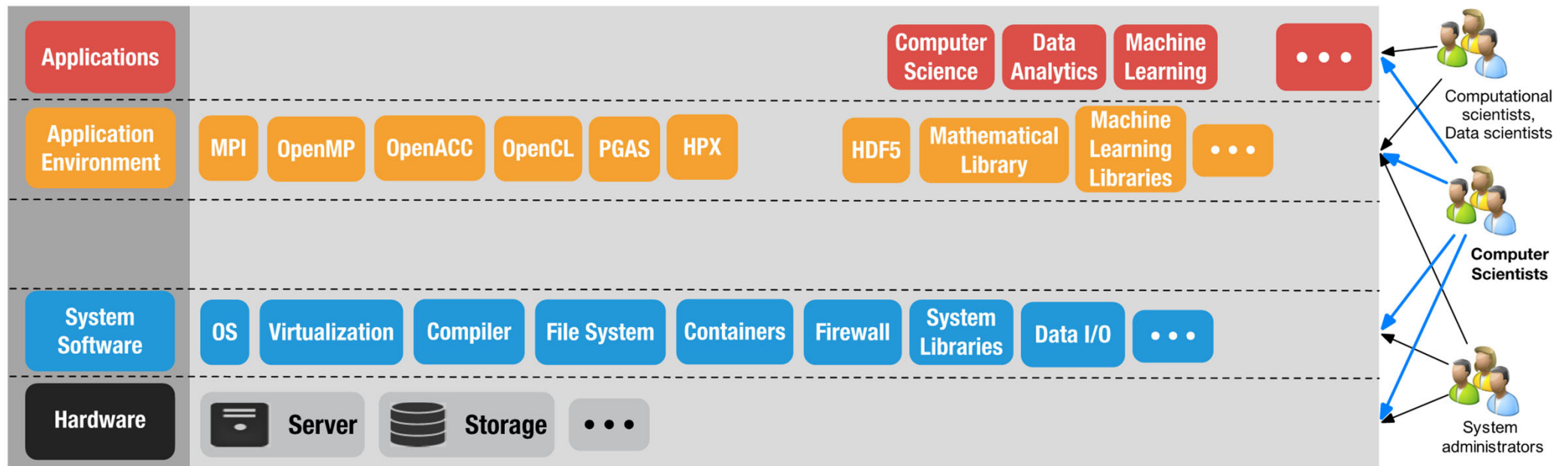
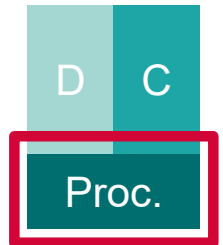
What are Systems that **Process Computations** (aka **Computing** Systems)?

- A **computer system** is the **ensemble** of the
 - Software (SW) and its architecture (SW-ARCH)
 - Hardware (HW) and its architecture (HW-ARCH)
- Characterized by the **interplay** between
 - Software components & architectures
 - Hardware components & architectures
 - System software & Tools
 - Applications
 - Services
- **Purpose**
 - Perform **computation** while seamlessly leveraging the available **data** and **communication** subsystems, under the threat of **privacy** violations.

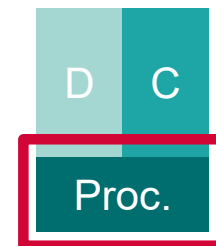
*“The purpose of computing is **insight**, not numbers.”*

R. Hamming
Numerical Methods for
Scientists and Engineers, 1973

Computing Ecosystem



What are **Parallel** & **Distributed Computing** Systems?

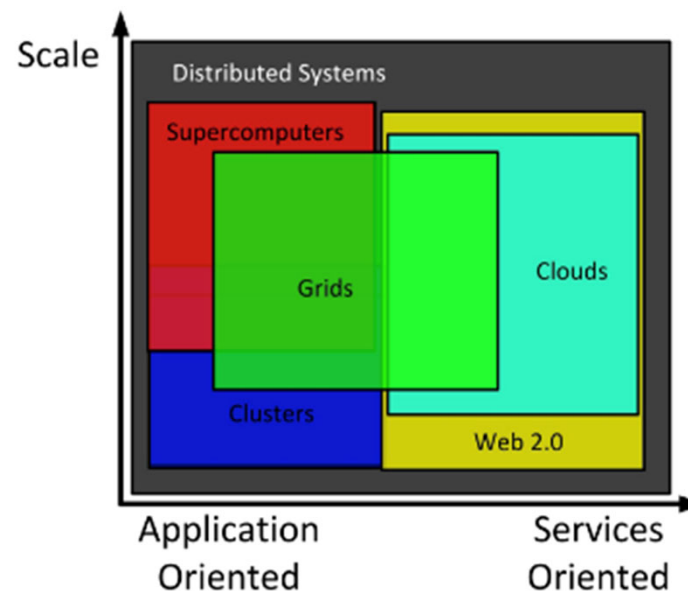


- **Parallel Computing** Systems (PCS)
 - Suitable for **tightly** coupled (synchronous) computations -> application oriented
 - Typically located within **geographical proximity**
 - With **shared** and/or distributed memory
 - Ex.: Clusters, Supercomputers

A **high performance computing (HPC) system** is a P&DCS system designed to *efficiently* solve a large problem, typically on more than a single computer (e.g., a Cluster, a Supercomputer, the Cloud).

A **supercomputer** is a **top class HPC** system concentrating the highest compute capacities on Earth: $\sim 10^4$ servers, 10^6 – 10^7 cores (mostly in accelerators), high-speed network.

- **Distributed Computing** System
 - Suitable for **loosely** coupled (asynchronous) computations -> services oriented
 - **Not** defined by geographical **proximity**
 - With shared and/or **distributed** memory
 - Ex.: Data Centers, Clouds, Grids



[Image: Ioan Raicu, Scalable Resource Management in Clouds and Grids, 2008]

High Performance **Parallel** & **Distributed** Computing

Processing Speed Magnitudes

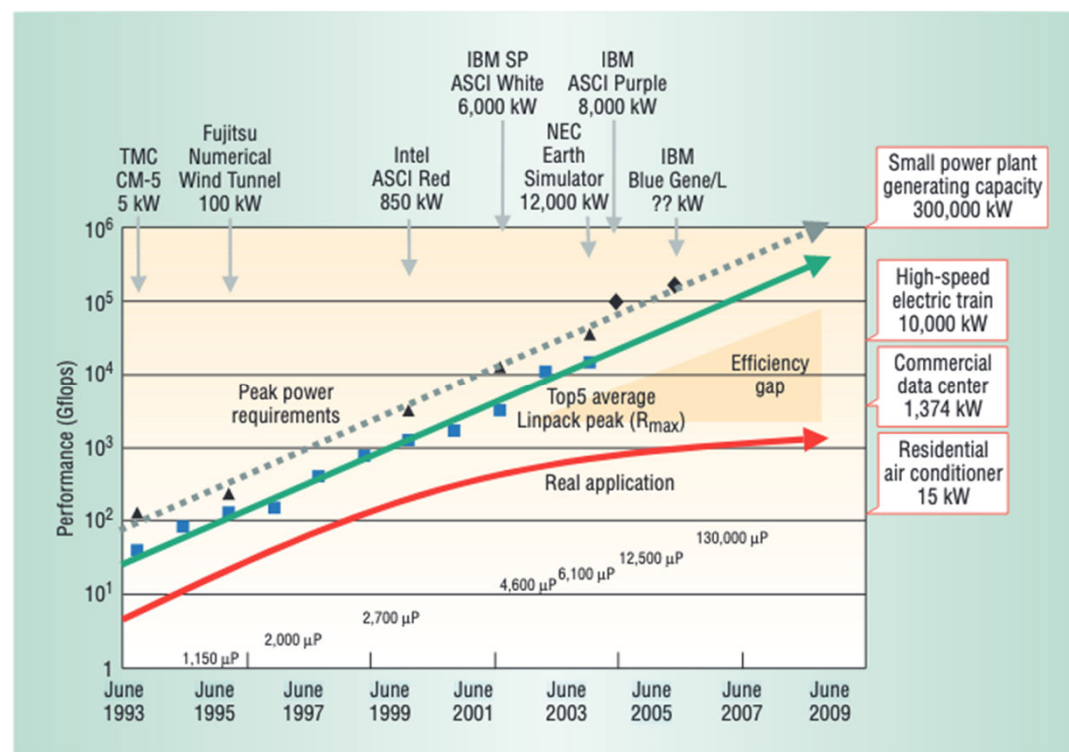
D C

Proc.

- Kilo = 10^3 , Mega = 10^6 , Giga = 10^9 , Tera = 10^{12} , Peta = 10^{15} , Exa = 10^{18} Flop/s
- **High Performance Computers** today
 - Laptop: GigaFlop/s on CPUs, TeraFlop/s on GPUs
 - Centres: Tera- to PetaFlop/s on CPUs+GPUs
 - **Leadership: ExaFlop/s since 2022**
 - 2005 forecast ~650 MW; 2026 reality 22–42 MW
 - Each increase in order of magnitude is a big deal:

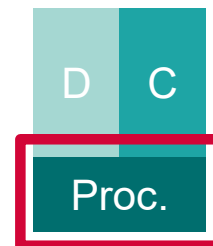
“[it is like going] from *walking* to explore one’s street, to *biking*, driving, and ultimately *flying* to explore the world”

(Dona Crawford)



[Image: Cameron, K. W., Rong Ge, & Xizhou Feng. High-performance, power-aware distributed computing for scientific applications. Computer 38, 40–47 (2005)]

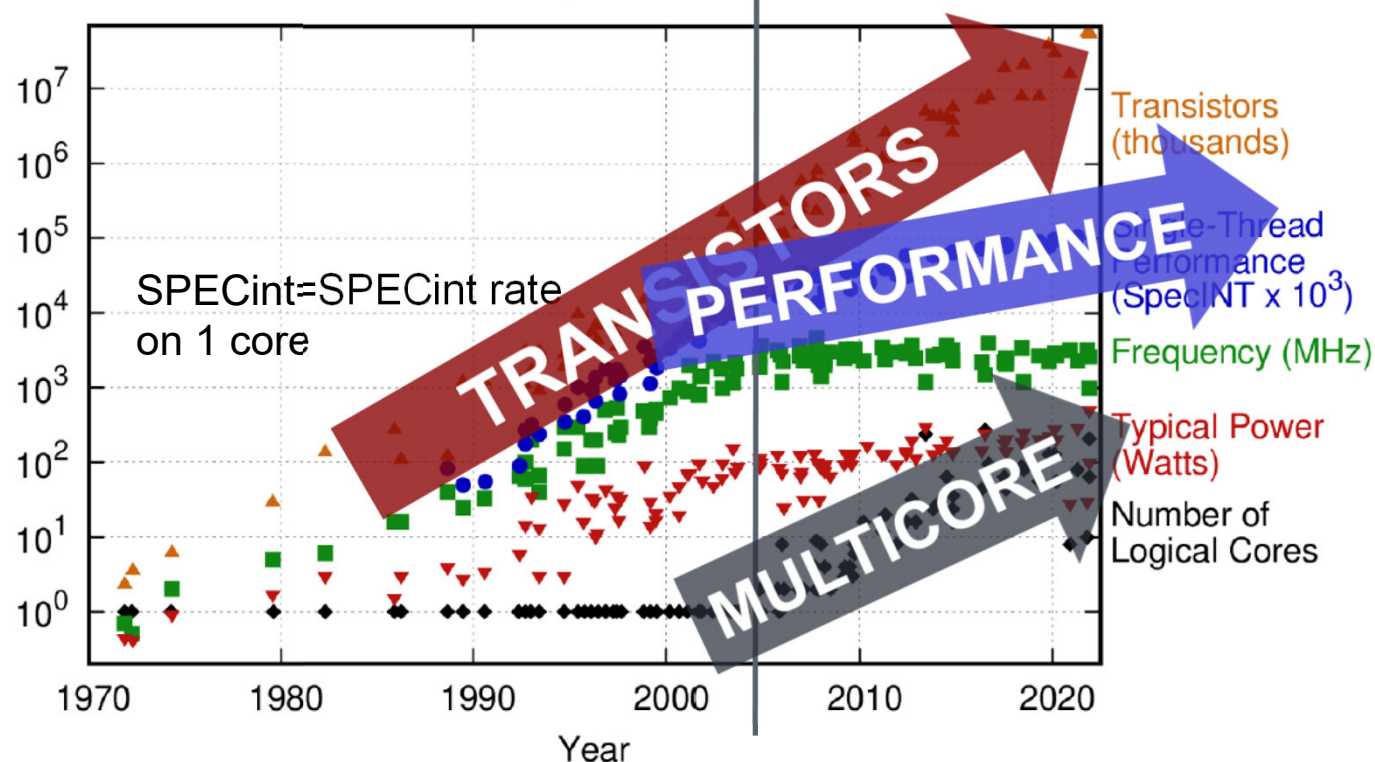
Evolution of HPC Systems over the last 50+ Years



Curves: transistors (thousands) · single-thread performance (SpecINT×10³) · frequency (MHz) · typical power (W) · logical cores

SPECint rate on 2+ cores

50 Years of Microprocessor Trend Data

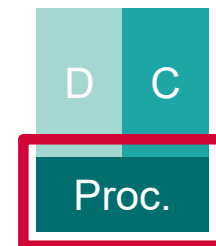


Original data up to the year 2010 collected and plotted by M. Horowitz, F. Labonte, O. Shacham, K. Olukotun, L. Hammond, and C. Batten
New plot and data collected for 2010-2021 by K. Rupp

[Adapted from Jack Dongarra, "HPC: Where We Are Today And A Look Into The Future", PPAM September 2022]

- **Moore's Law (1965/75):** transistor count doubles $\approx$ every 2 years
 - **Dennard Scaling (1974):** smaller transistors $\rightarrow$ lower voltage & current $\rightarrow$ constant power density
 - **End of Dennard Scaling (2004):** leakage & heat end voltage scaling $\rightarrow$ single-core perf. plateaus
- $\rightarrow$ **Multicore era (2005–today):** performance from parallelism, not clock speed

The Power Wall: why ExaFlop/s took 15 years

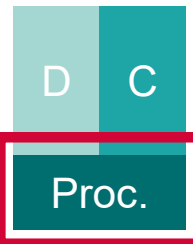


- Until ~2004, Dennard scaling gave performance for free: shrink, raise the clock, keep power density
- After it ended, **every extra Flop/s costs extra watts** → power, not transistors, became the constraint
- Naive 2005 extrapolation put an exascale machine at **~650 MW**, i.e. $\approx 2\times$ a small power plant (previous slides)
- The community set itself a **20 MW target for 1 EF/s** → a $\sim 30\times$ gain in Flop/s per watt
- As of June 2026: **LineShine** 42.2 MW (#1 on Top500), **El Capitan** 29.6 MW (#2), **Frontier** 22.8 MW (#3)
 - **Accelerators closed the gap**: trade single-thread speed for parallelism at low clock and low voltage
 - Green500 leader: KAIROS, 73.3 GFlop/s per watt

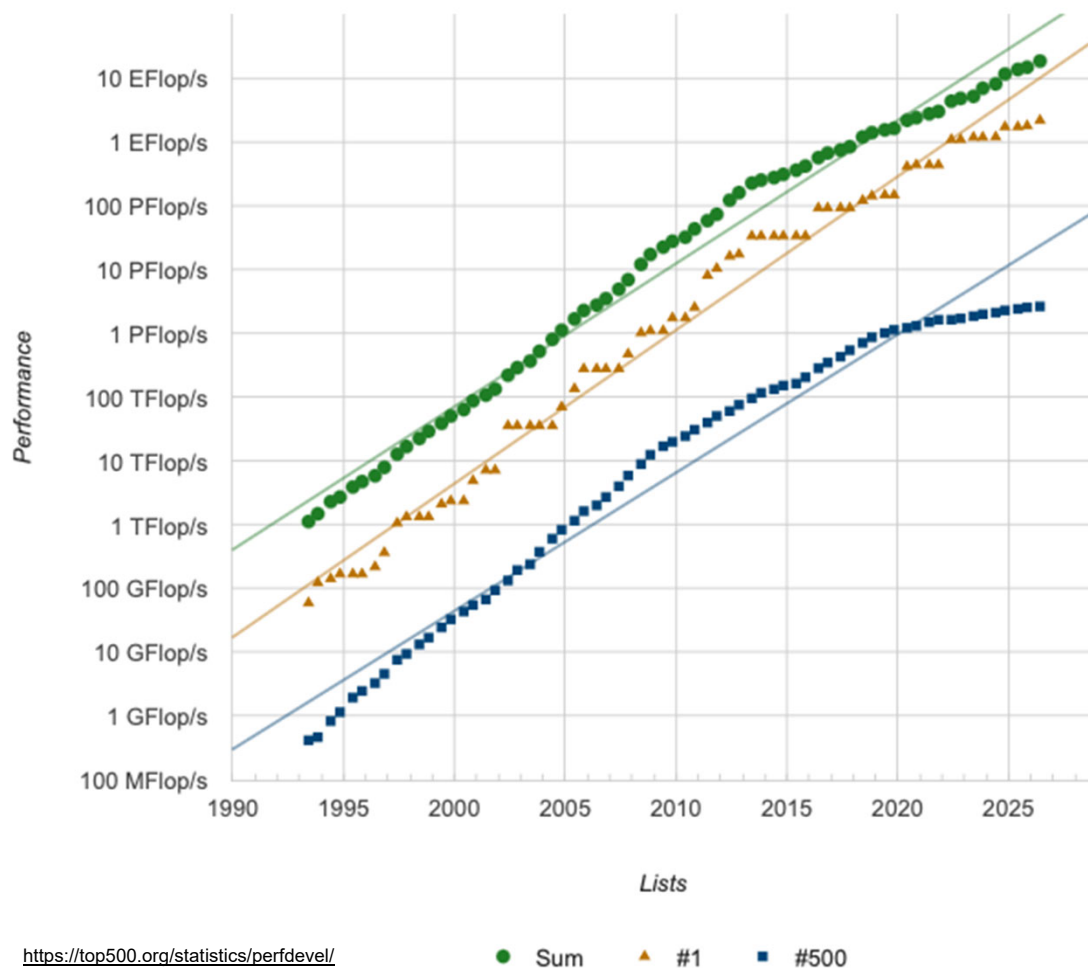
Today, an HPC program is written against an **energy budget**, not only a time budget

- **Food for thought**: your code runs $2\times$ faster but draws $3\times$ the power ☐ do you call this an improvement? If yes, in terms of what?

Evolution of HPC Systems Performance over the last 20+ Years



Projected Performance Development



- **Speed Records:** measured w/ HPL benchmark
→ TOP500 list, twice a year



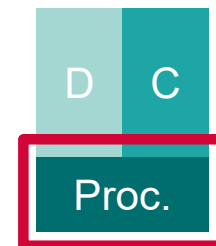
- **Exascale Supercomputer: Frontier @ORNL,**
June 2022: **1.102×10^{18} Flop/s HPL**
= one quintillion operations / second

TOP500 June 2026 (67th list):

- #1 **LineShine** (Shenzhen) **2.198 EF/s, no GPUs!**
- #2 **El Capitan** (LLNL) 1.809 EF/s
- #3 **Frontier** (ORNL) 1.353 EF/s (was upgraded)
- #4 **Aurora** (ANL) 1.012 EF/s
- #5 **JUPITER** (Jülich) 1.000 EF/s – Europe's first
- ...
- #10 **Alps** (CSCS) 434.9 PF/s – Swiss flagship

<https://top500.org/lists/top500/2026/06/>

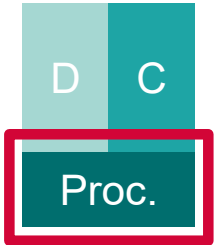
One Number Is Not Enough: HPL, HPCG and HPL-MxP



- **HPL** (Linpack) solves a dense linear system: compute-bound, near-perfect locality → it flatters the machine 🤔
- **HPCG** (High Performance Conjugate Gradient) uses a sparse multigrid solver: memory- and communication-bound → it behaves like real simulations 🤔
- **HPL-MxP** repeats HPL in **mixed** / low precision → a proxy for AI training workloads (which require less than 32 or 64 bits)
- Same machines but three different rankings (June 2026):
 - #1 **LineShine**: **HPL 2.198 EF/s**, **HPL-MxP 7.92 EF/s** (3.6× over HPL), but **HPCG only 22.0 PF/s** → about 1% of its HPL number
 - LineShine gains only 3.6× on HPL-MxP due to being CPU-only (LX2 processor), no low-precision units
 - #2 **El Capitan**: **HPL 1.809 EF/s**, **HPL-MxP 16.7 EF/s** (9.2× over HPL), but **HPCG only 17.4 PF/s** → a bit more than 1% of its HPL number
- This is the “**efficiency gap**” between Linpack peak and real applications on earlier 2005 figure (Cameron et al.)

Take-home: “fastest supercomputer” is benchmark-relative → which benchmark resembles your code?

Exascale Supercomputing Era

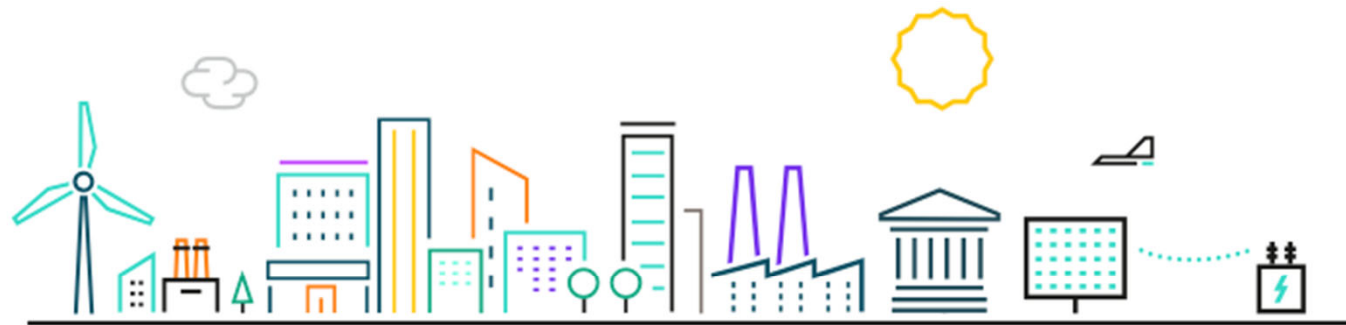


Start counting!

.....

If someone started counting one number per second, it would take almost 32 billion years to **count to one quintillion!**

What can we do with exascale supercomputers?



Weather

Precise weather prediction

Climate

Better understand climate change

Clean Energy

Wind, ocean, and fusion energy development

Life Sciences

Personalized, predictive healthcare

Manufacturing

Creating safer, more sustainable products

Government

Creating smarter cities and utilities

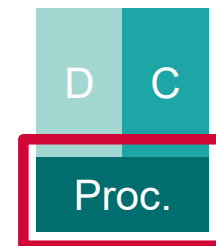
*“The purpose of computing is **insight**, not numbers.”*

R. Hamming

Numerical Methods for Scientists and Engineers, 1973

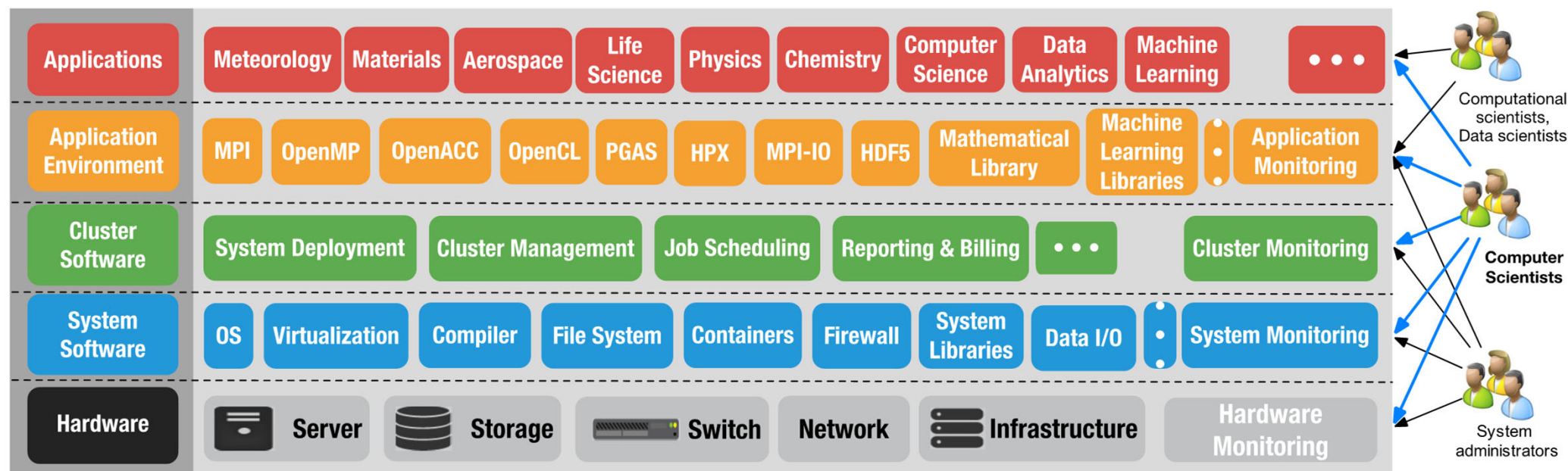
High Performance Computing | Supercomputing

Rich, Diverse, Complex Ecosystem



- “*Far more than* one sponsor, one site, one application, one software system, or one architecture. [...] it is a **community of interacting entities**.”

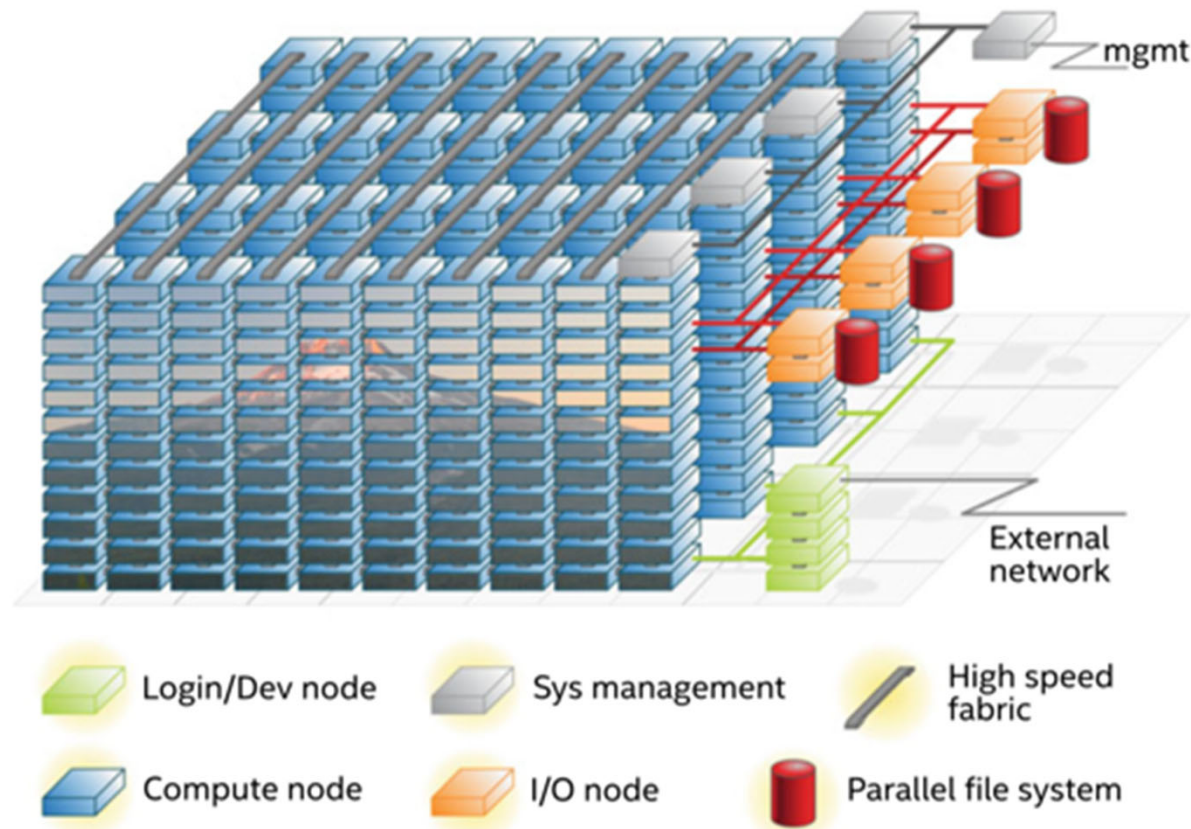
“Contemporary High Performance Computing: from Petascale toward Exascale”, 2019, p. xix



High Performance Computer | Supercomputer

A Parallel & Distributed System of Subsystems

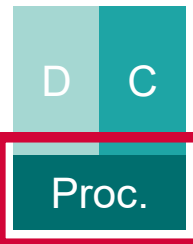
- **On-premises:**
dedicated nodes,
low-latency fabric,
batch scheduler,
parallel file system
- **In the Cloud:**
virtualized,
elastic,
pay-per-use
commodity network,
higher latency & OS jitter



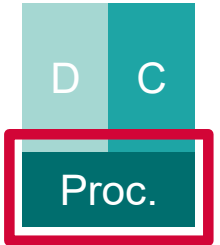
Not in this figure:
accelerator (GPU/APU) nodes
□ 8 of the June '2026 TOP10
systems get most of their Flop/s
from accelerators, not CPUs.

Fig. 1.2 Depiction of major, common subsystem partitioning in large HPC machines

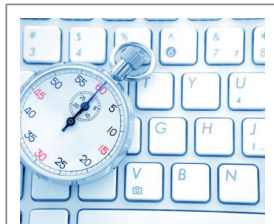
[Source: B. Geroft et al. (eds.), Operating Systems for Supercomputers and High Performance Computing, 2019]



From a **Processing** Perspective: Goals & Challenges of Distributed Systems



• Goals



Performance

```
00111101001010101
01001010100010101
01010ERROR011010
10100010100110001
01100010100100111
10100010100110001
```

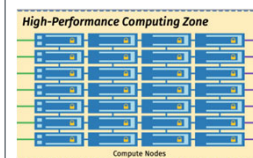
Resiliency



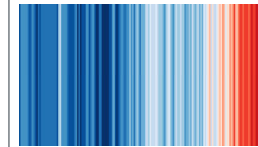
**Extreme
scalability**



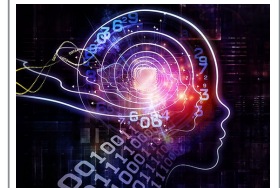
Energy



Security

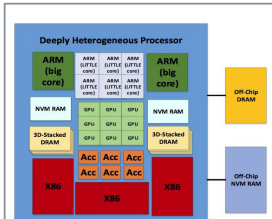


Sustainability

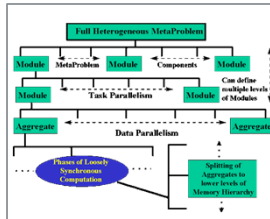


**Convergence
HPC+Data+AI**

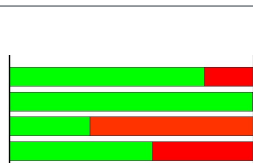
• Challenges



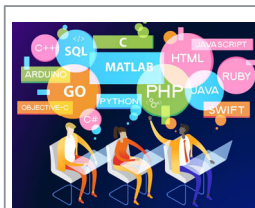
Heterogeneity



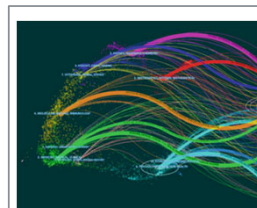
**Multilevel
parallelism**



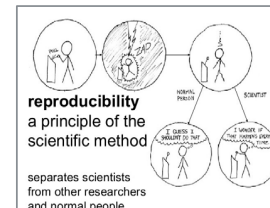
Load imbalance



Programming

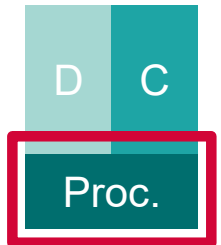


Visualization

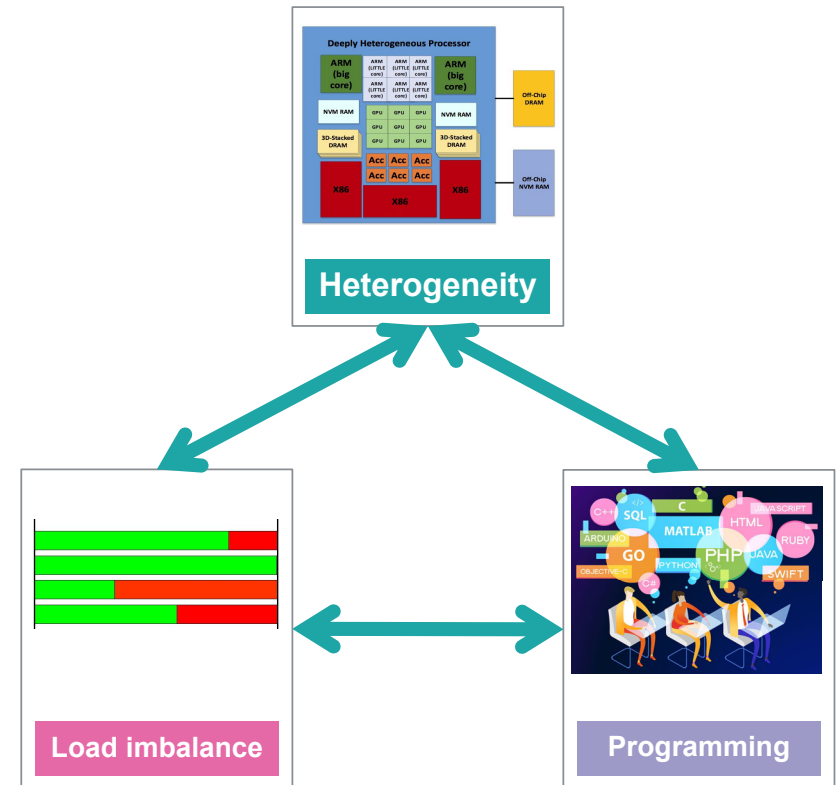
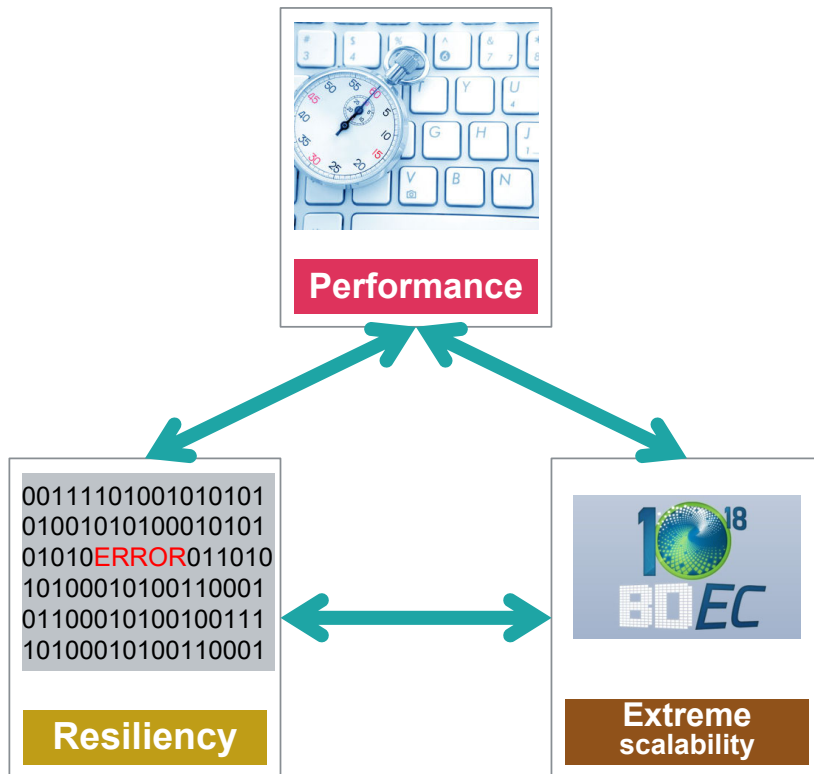


Reproducibility

Dependencies



- Some of these goals and challenges lead to tug-of-war situations **requiring trade-offs**
- **Performance** is the currency; **HPC** is a **trade-off space**



Dependencies

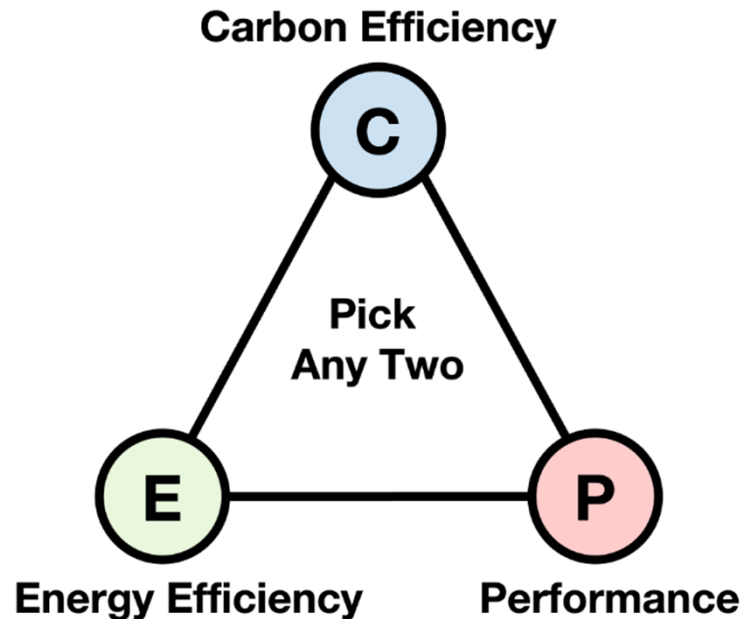
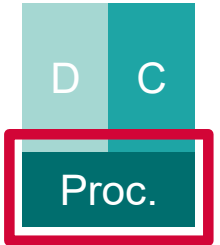


FIGURE 4. The CEP impossibility conjecture: “Pick any two of carbon efficiency, energy efficiency, and performance”.

- **CEP Impossibility Conjecture**
- Under conditions where
- Energy is **not entirely** carbon-neutral and
- Carbon intensity **fluctuates** across **time** and **space**
- It is **unattainable** for systems to **concurrently optimize**
- **Carbon-efficiency**,
- **Energy-efficiency**, and
- **Performance**.

A diagram showing a 2x2 matrix. The top-left cell is light blue and contains the letter 'D'. The top-right cell is teal and contains the letter 'C'. The bottom row is a single teal cell containing the text 'Proc.'. A red rectangular box highlights the entire bottom row, including the 'Proc.' cell.

Chapter 5.2: Parallel architectures & models



Performance

```
00111101001010101
01001010100010101
01010ERROR01101010
10100010100110001
01100010100100111
10100010100110001
```

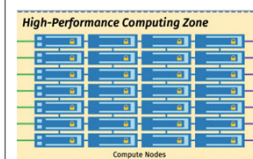
Resiliency



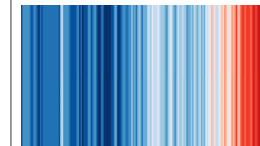
Extreme scalability



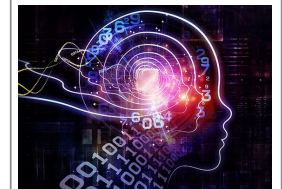
Energy



Security

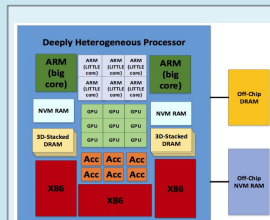


Sustainability

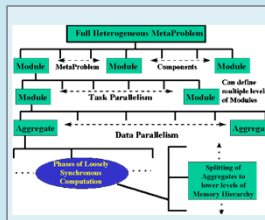


**Convergence
HPC+Data+AI**

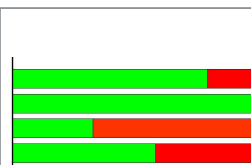
Chapter 5.3: Parallel programming



Heterogeneity



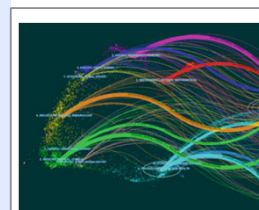
Multilevel parallelism



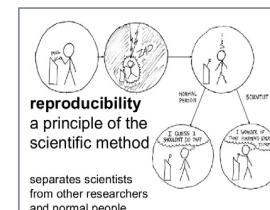
Load imbalance



Programming



Visualization



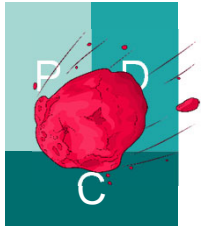
Reproducibility

Overview of Chapter 1

-
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Data Perspective |
| 1.4 | ... from a Processing Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |
-

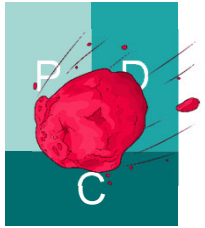
1.5 What are Distributed Systems?

An Answer from a **Privacy** Perspective

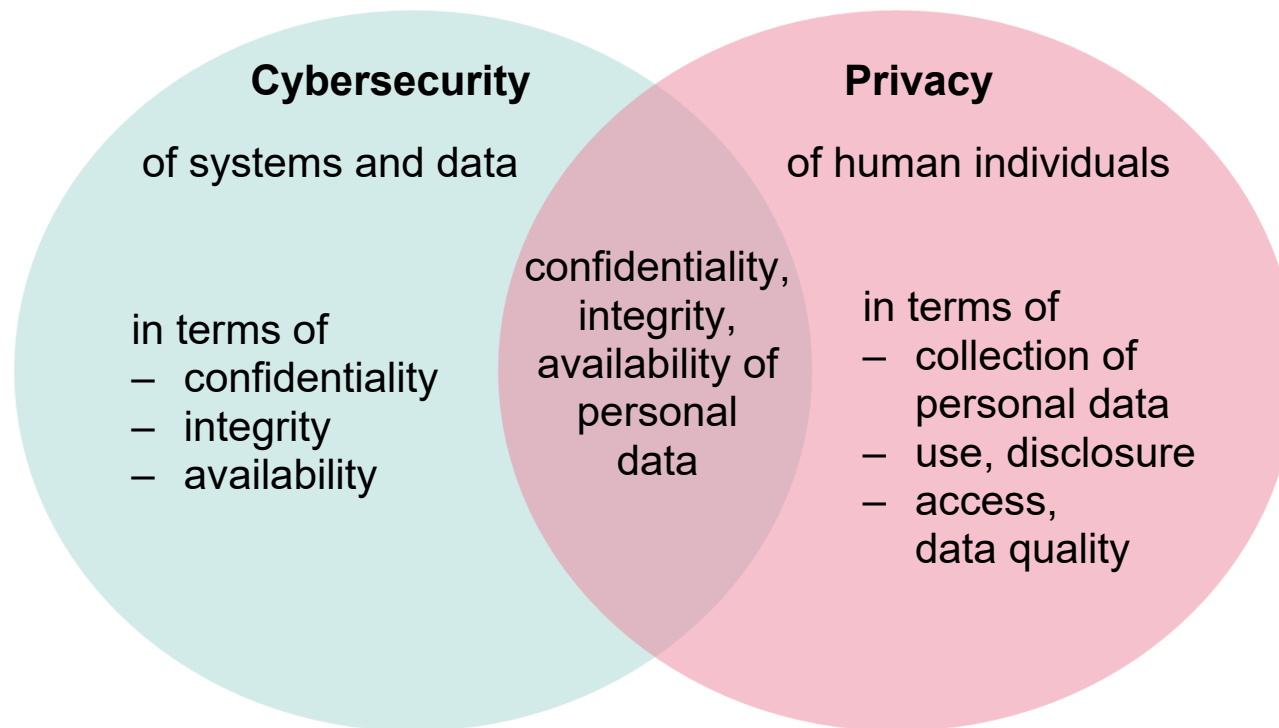


- Privacy is a **cross-cutting concern** in distributed systems
 - It affects all aspects of distributed system design: handling of data, design of communication, realization of processing
- Privacy is a **non-functional property**
 - To make a distributed system work, you absolutely have to figure out data, processing, and communication
 - But you can get away with disregarding privacy and still have a “working” system
- There are other cross-cutting non-functional properties (“asteroids”) besides privacy:
 - Security
 - Transparency
 - Usability
 - Environmental protection
 - ... And many more (see https://en.wikipedia.org/wiki/Non-functional_requirement)

Privacy vs Security

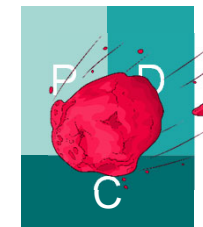


- We focus on privacy because it is a fundamental human right
- There is some overlap with security (but only some!)



Privacy implications of cheap storage

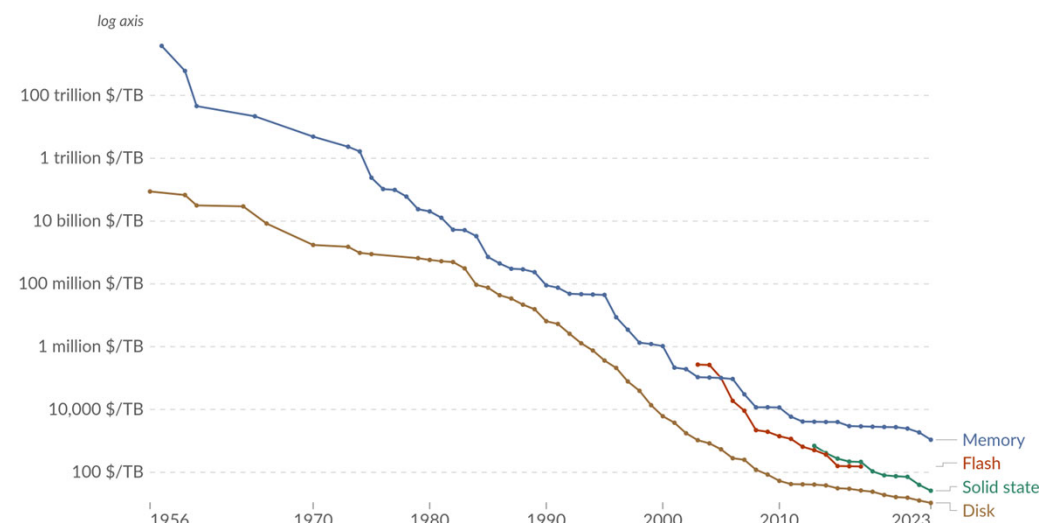
- Decrease in storage cost leads to increased amount of data stored
- What kinds of data can now be stored without thinking twice, and essentially forever?
 - User behaviors, e.g., click traces, location history
 - Transaction histories
 - Video streams from surveillance cameras
 - History of search terms and resulting clicks
 - All video frames from your electric (self-driving) car



Our World
in Data

Historical price of computer memory and storage

This data is expressed in US dollars per terabyte (TB), adjusted for inflation. "Memory" refers to random access memory (RAM), "disk" to magnetic storage, "flash" to special memory used for rapid data access and rewriting, and "solid state" to solid-state drives (SSDs).

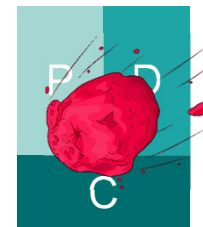


Data source: John C. McCallum (2023); U.S. Bureau of Labor Statistics (2026)

OurWorldinData.org/technological-change | CC BY

Note: For each year, the time series shows the cheapest historical price recorded until that year. This data is expressed in constant 2020 US\$.

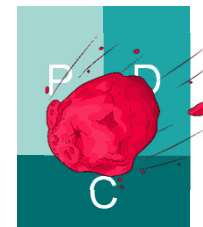
Privacy implications of big data



- Facebook
 - approx. 350 million of new photos per day
 - More than 300 petabytes of data
 - Approx. 100 million hours of video are watched every day
- Who owns these photos/videos?
- Who can see them, and are users aware of this? (friends, friends of friends, the public)
- What does Facebook itself do with the photos?
 - Training of face recognition models and other AI?
 - Extract information from subjects and backgrounds?
- Does Facebook sell these models and inferred information?
- Photos/videos of me can be uploaded without my knowledge



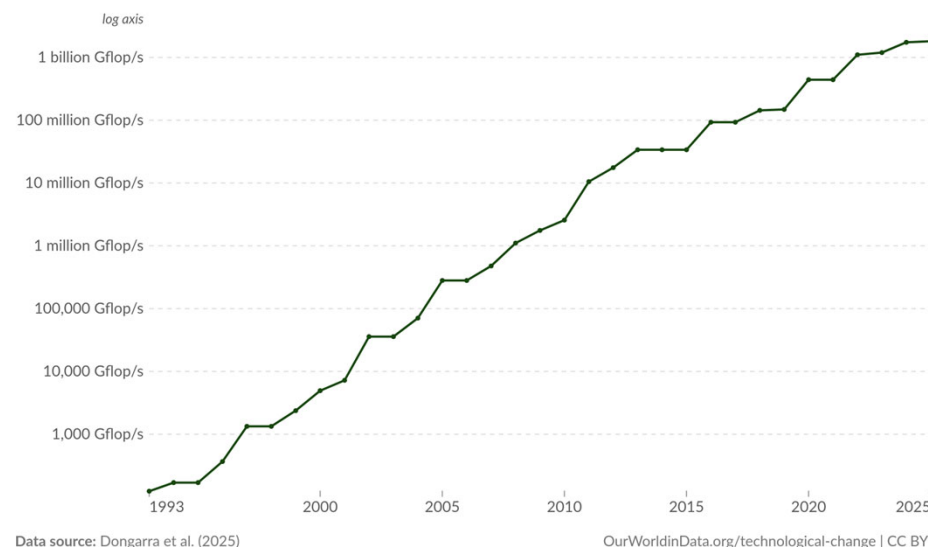
Privacy implications of increased compute power



- Increase in compute capabilities means the stored data can be subjected to ever more analysis
- Inferences made by Facebook, based on user behavior, views/clicks (Kosinski 2013, PNAS):
 - sexual orientation
 - ethnicity
 - religious and political views
 - personality traits
 - intelligence
 - happiness
 - use of addictive substances
 - parental separation
 - age
 - gender

Computational capacity of the fastest supercomputers

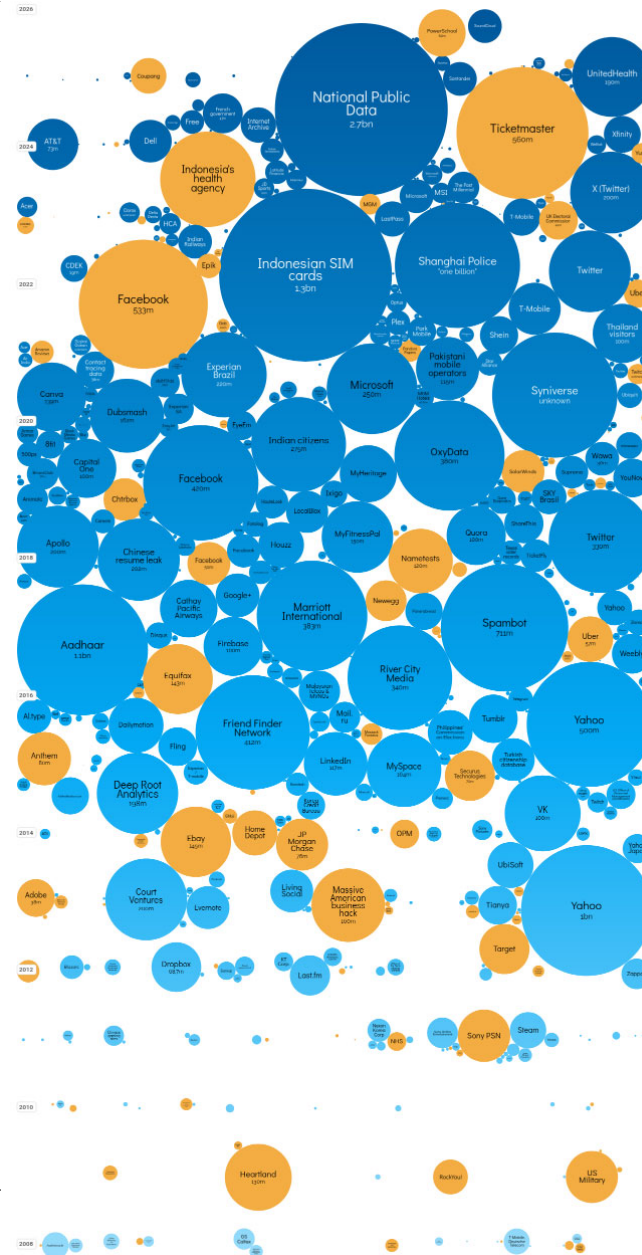
Number of floating-point operations¹ carried out per second by the fastest supercomputer in any given year. This is expressed in gigaflops, equivalent to 10^9 floating-point operations per second.



1. Floating-point operation A floating-point operation (FLOP) is a type of computer operation. One FLOP represents a single arithmetic operation involving floating-point numbers, such as addition, subtraction, multiplication, or division.

**more compute
+ cheaper storage
= more data breaches?**

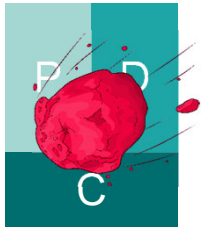
2026



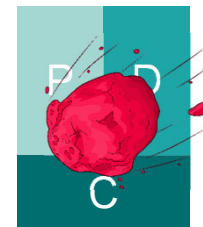
Source: <http://www.informationisbeautiful.net/visualizations/worlds-biggest-data-breaches-hacks/>

Fall 2026

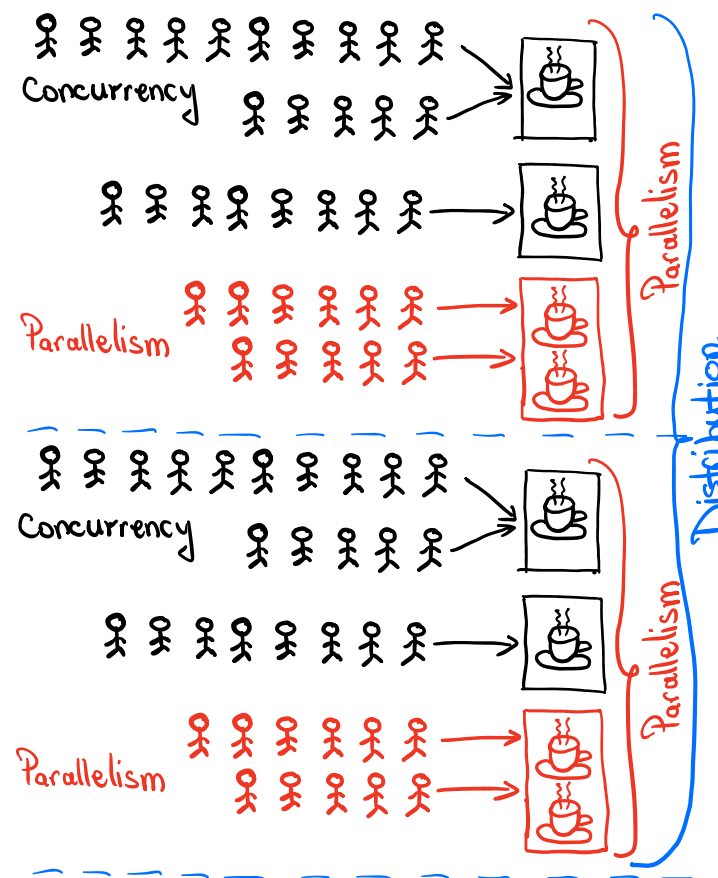
2006



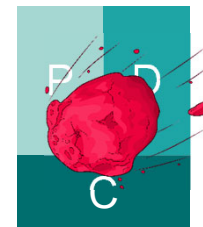
Privacy implications of (outsourced) computing



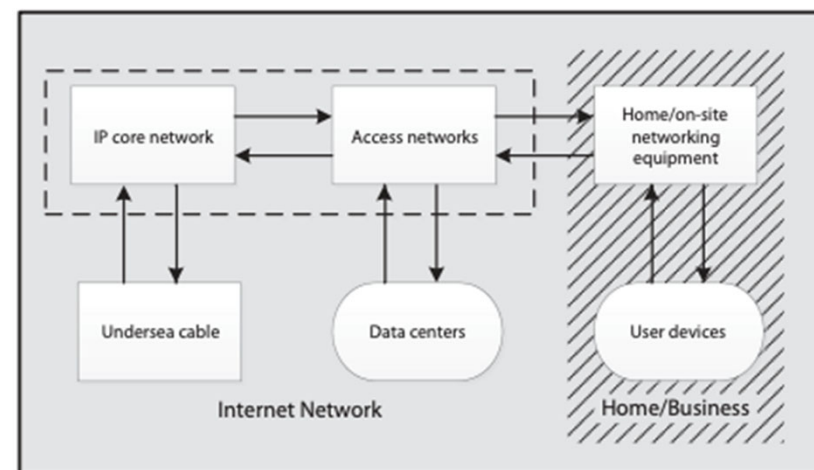
- What if the compute provider faithfully executes computations, but tries to learn additional information?
- What can a computing provider learn from observing compute jobs?
 - How often does someone enter the coffee queue?
 - At what times of day?
 - What drink does someone prefer?
 - At which geographic locations does someone use coffee queues?
 - What other patterns can we observe?
 - Are two users often served “together”?



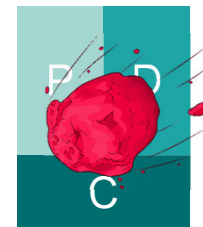
Privacy implications of networked communication



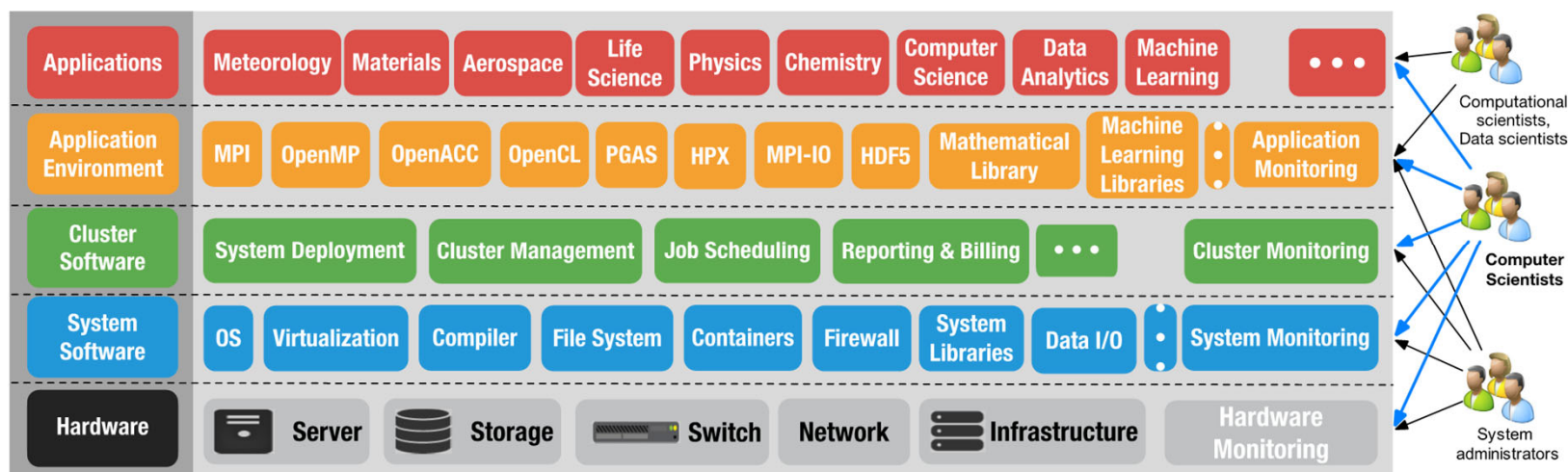
- Who operates the Internet network equipment, and what information can they learn?
 - What if someone can observe traffic on an undersea cable?
 - On my wireless connection?
 - At my internet provider's access network?
- Confidentiality of traffic content: very common, implemented with modern cryptographic tools (such as TLS)
- Privacy of metadata: much harder
 - IP addresses are fairly unique identifiers, change infrequently
 - IP addresses can be geolocated, provider can link to user identity



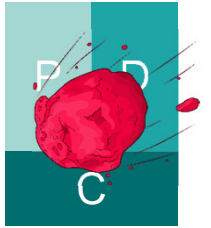
How can we address privacy threats?



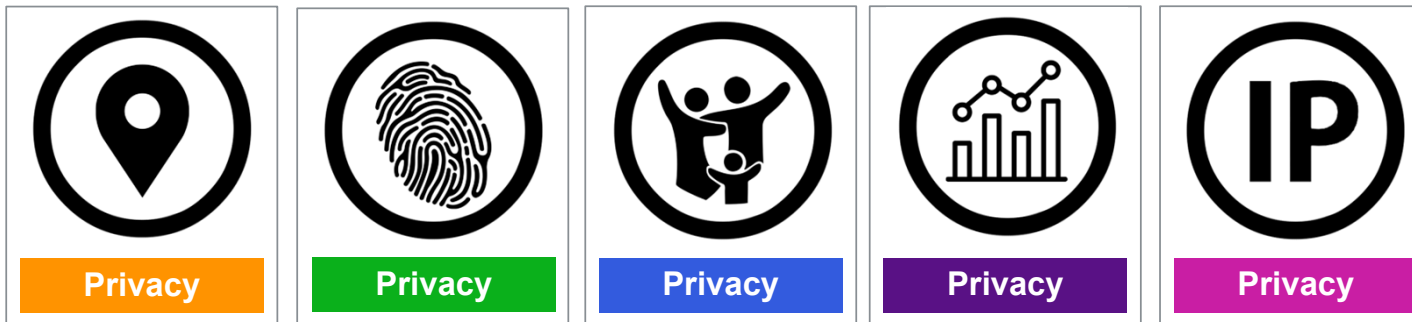
- Privacy mechanisms exist on all layers
- Abstractions are useful: lower-layer mechanisms provide **privacy-as-a-service** to higher layers
- But: problems and unintended side-effects often occur at the interfaces between layers
 - Do we understand the abstraction provided by the lower layer?
 - Is it the right abstraction for our application?



From a Privacy Perspective: Goals & Challenges of Distributed Systems



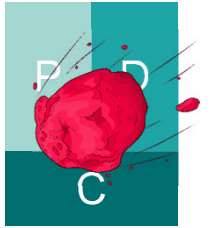
- Goals



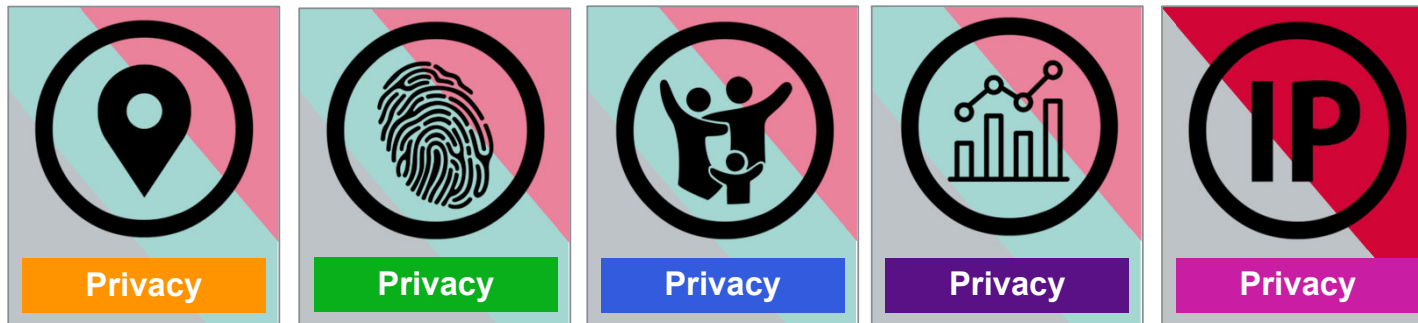
- Challenges



From a Privacy Perspective: Goals & Challenges of Distributed Systems



- Goals



- Challenges



Chapter 6.1: Introduction

Chapter 6.2: Differential privacy

Chapter 6.3: Homomorphic encryption

Chapter 6.4: Anonymous communication

Overview of Chapter 1

-
- | | |
|-----|---|
| 1.1 | What are parallel and distributed systems? |
| 1.2 | ... from a Communication Perspective |
| 1.3 | ... from a Data Perspective |
| 1.4 | ... from a Processing Perspective |
| 1.5 | ... from a Privacy Perspective |
| 1.6 | Again: what are Parallel and Distributed Systems? |

1.6 Again: What are the Challenges in Parallel and Distributed Systems?

Mastering Concurrency

- Align the goals and challenges ... from three different angles
- Work against impossibility theorems
- Carefully choose between conflicting goals, be aware of the consequences
- Have the right mindset!

Bachelor curriculum ~ (mostly) sequential programs

Master curriculum ~ (increasingly) concurrent, parallel, distributed programming

- Engineering choices and “Zeitgeist”:
Central control vs. decentralized operations